

AdaptiveQKE: Context-Aware Hybrid Key Exchange for TLS Channels

by

Heba Abd El-Salam Ajjour
Weaam Raed Abdalaal
Hala Ashraf Nasrallah
Farah Hussein Rajab

A dissertation submitted in partial fulfillment of the
requirements for the degree of
Bachelor of Cyber Security Engineering(Computer Engineering)

in the

Computer Department
of the
University College of Applied Sciences (UCAS), GAZA

Graduation Projects advisor:
Eng. Hadi El-Nabris

(Aug 2026)

AdaptiveQKE: Context-Aware Hybrid Key Exchange for TLS Channels

Copyright © Aug 2026

by

Heba Abd El-Salam Ajjour

Weaam Raed Abdalaal

Hala Ashraf Nasrallah

Farah Hussein Rajab

Abstract

TLS 1.3 deployments adopting post-quantum hybrid key exchange typically enforce a single hybrid group across every connection, regardless of the wide variation in endpoint capability and network conditions a given connection may encounter. This work presents AdaptiveQKE, a client-side, policy-driven mechanism that selects among ML-KEM and ECDH hybrid groups before the handshake begins, based on measured round-trip time and endpoint resource limits, without modifying the standardized TLS 1.3 handshake flow.

AdaptiveQKE was implemented as a context probe, a two-stage policy engine, and a TLS client operating against an unmodified server, and evaluated across nine device–network conditions using 1,800 TLS handshake measurements spanning three static hybrid configurations and the adaptive policy. Group selection matched the policy's decision rules in all nine tested conditions, and once a group was selected, its handshake behaved indistinguishably from the corresponding static configuration. Relative to the strongest static baseline, AdaptiveQKE reduced handshake cost under constrained network and device conditions while retaining the strongest group when conditions supported it, at the cost of a measurable, network-dependent probing delay on first connection.

These results indicate that context-aware group selection is a viable complement to static configuration in heterogeneous TLS deployments, particularly where device or network constraints vary enough to make a fixed configuration inefficient. The evaluation used emulated device and network profiles on a single host; a conceptual downgrade-resistance stage was designed but not implemented, and no formal security analysis was performed. Within this scope, the work contributes a working selection layer, an empirical account of the relationship between handshake size, link rate, and device constraint, and a reproducible methodology for evaluating adaptive TLS mechanisms.

Acknowledgments

All praise and gratitude are due to Almighty Allah, whose grace enabled us to complete this graduation project. Without His blessings, this achievement would not have been possible.

We express our deepest gratitude to our advisor, Eng. Hadi El-Nabris, for his guidance, constructive feedback, encouragement, and continuous support throughout the development of AdaptiveQKE.

We also extend our appreciation to the academic community of the University College of Applied Sciences (UCAS) for their knowledge and guidance. We solemnly honour the memory of the martyred Dr. Saeed Al-Zebda, former President of UCAS, and all the martyred academics of the College who devoted themselves to education and their students. Their dedication and lasting legacy will remain a source of pride and inspiration. May Almighty Allah have mercy upon them, accept them among the martyrs, and grant them the highest ranks in Paradise.

Our special appreciation goes to the current President of UCAS, Dr. Mohammed Mushtaha, a professor and steadfast supporter whose teaching and encouragement enriched our academic journey. We also thank all our professors, lecturers, and instructors for contributing to our academic and professional development.

We recognise the collective effort of our team: Heba, Weaam, Hala, and Farah. Each member contributed her knowledge, ideas, and dedication. Through shared responsibility, mutual respect, and continuous support, we overcame challenges and worked toward a common purpose. This project is a shared accomplishment, and every contribution was valuable and essential.

Finally, our heartfelt gratitude goes to our families for their unconditional love, prayers, patience, and extraordinary sacrifices. Despite the unstable circumstances caused by the war on Gaza, they did everything possible to provide suitable conditions for us to continue our education and complete this project. Their support gave us strength through our most difficult moments. To them, we dedicate this achievement with our deepest love and appreciation.

Contents

Abstract **III**

Acknowledgments **IV**

List of Tables **4**

List of Figures **6**

1 Introduction **7**
1.1 Background . . . 7
1.2 Problem Statement . . . 9
1.3 Objectives . . . 10
1.3.1 Design and Implement a Context-Aware Hybrid Group Selection Layer for TLS 1.3 . . . 10
1.3.2 Evaluate Adaptive Selection Against Static Hybrid Configurations Across Device and Network Conditions . . . 10
1.4 Project Scope . . . 10
1.5 Project Significance . . . 11
1.6 Report Structure . . . 12

2 Literature Review **14**
2.1 Introduction . . . 14
2.2 Theoretical Background . . . 15
2.2.1 TLS and Cryptographic Primitives . . . 15
2.2.2 Quantum Computing and Post-Quantum Cryptography . . . 15
2.2.3 Hybrid Key Exchange in TLS 1.3 . . . 16
2.2.4 Adaptive and Context-Aware Selection . . . 17
2.3 Related Works . . . 17
2.4 Gap Analysis . . . 20
2.5 Summary . . . 21

3 Design & Methodology **22**
3.1 Introduction . . . 22
3.2 Overall Methodology . . . 23
3.2.1 Phase 1 — Design and Implementation (Objective 1) . . . 23
3.2.2 Phase 2 — Evaluation and Analysis (Objective 2) . . . 25

3.2.3 Justification of Design Choices 26
3.3 Project Design 27
3.3.1 System Architecture 27
3.3.2 Hybrid Key Exchange Construction 29
3.3.3 Policy Engine 33
3.3.4 Experimental Setup 43
3.4 Performance Metrics 52
3.4.1 TLS Handshake Latency 52
3.4.2 CPU Time 54
3.4.3 TLS Handshake Size 55
3.4.4 AdaptiveQKE Performance Testing Metrics 57
3.5 Summary 57

4 Results & Analysis **59**
4.1 Introduction 59
4.2 Experimental Execution 60
4.3 Data Validity Checks 61
4.4 Selection Accuracy and Policy Overhead 63
4.4.1 Selection Accuracy 63
4.4.2 Policy Overhead 64
4.4.3 Total First-Connection Cost 65
4.5 Post-Selection Consistency with the Matching Static Configuration . 67
4.6 Group Latency across Network Conditions 69
4.7 Separating Network Delay from Key-Exchange Cost 72
4.8 Handshake Cost without Network Delay 74
4.9 Effect of Device Constraints 76
4.10 Adaptive vs. the Static Baselines 78
4.11 Summary 82

5 Conclusions and Future Work **83**
5.1 Introduction 83
5.2 Achievement of Objective 1 83
5.3 Achievement of Objective 2 85
5.4 Contributions 86
5.5 Deployment Implications 87
5.6 Limitations 88
5.7 Future Work 90
5.8 Concluding Remarks 90

A Reproducibility and Supporting Materials **92**
A.1 Purpose of the Appendix 92
A.2 AdaptiveQKE Implementation Components 92
A.3 Adaptive Selection Implementation 93
A.4 Experimental Profile Enforcement 94
A.5 Context Collection and Performance Measurement 94
A.6 Experimental Output 95
A.7 TLS Experimental Configuration 96

A.8 Integrity Verification 97

Bibliography **98**

List of Tables

2.1 Comparison of related works and the proposed AdaptiveQKE framework 19

3.1 Hybrid Key Exchange Parameters, all sizes in bytes 33
3.2 RTT-Driven Candidate Determination 35
3.3 Device-Aware Candidate Validation 36
3.4 Policy Engine Threshold Values and Rationale 41
3.5 Simulated device profiles 44
3.6 Simulated network condition profiles and parameters 46
3.7 Experimental environment specifications 48
3.8 Summary of performance metrics used to evaluate AdaptiveQKE . . . 57

4.1 Summary of dataset validation checks 61
4.2 Relationship between configured packet loss and statistical outliers . . 62
4.3 Adaptive policy selection accuracy across device–network conditions . 63
4.4 Context probing and policy evaluation overhead across network conditions (all time values in ms) 64
4.5 Total first-connection cost across device–network conditions, with all latency values reported in milliseconds 65
4.6 AdaptiveQKE wire latency compared with the matching static configuration, separated by device profile. Differences are calculated as AdaptiveQKE minus static; the intervals are 95% bootstrap confidence intervals for the median difference. 68
4.7 Observed Static-768 minus Static-512 wire-latency gap on the high-performance device, by link rate. 70
4.8 Residual wire latency after subtracting the modeled propagation component on the high-performance device. 72
4.9 Baseline TLS handshake latency under the High-performance profile without emulated network delay ($n = 50$ per group). 74
4.10 Median Wire Latency Across Device Profiles Under the Low-Latency Network Profile. 76
4.11 Crypto-Phase Handshake Latency Across Device Profiles, with Enforcement Evidence 76
4.12 Adaptive Savings Relative to SecP384r1MLKEM1024 Across Device and Network Profiles 78
4.13 Adaptive Policy Compared with the Three Static Hybrid Configurations Across the Tested Device-Resource and Network Profiles 80

A.1 Main AdaptiveQKE implementation components 93
A.2 SHA-256 hashes of frozen implementation and dataset files 97

List of Figures

3.1 High-level overview of the AdaptiveQKE two-phase methodology . . . 23
3.2 Phase 1 workflow of the adaptive hybrid TLS mechanism 25
3.3 System architecture of the proposed AdaptiveQKE framework 29
3.4 Classical and Post-Quantum Key Exchange Mechanisms 30
3.5 Hybrid TLS 1.3 Handshake Protocol Flow 32
3.6 AdaptiveQKE policy engine decision flowchart 39
3.7 Experimental workflow across profile combinations and configurations 50

4.1 Observed Static-768 minus Static-512 wire-latency gap on the high-performance device, by link rate; error bars show 95% confidence intervals. 70
4.2 Residual wire latency by key-exchange group and link rate on the high-performance device. The link-rate axis uses a logarithmic scale. . 73
4.3 Crypto-phase handshake latency on the Constrained device, by hybrid group. Dotted lines mark multiples of the 10 ms CFS scheduling period. 77
4.4 Wire-latency savings achieved by Adaptive relative to `SecP384r1MLKEM1024` across the nine tested device and network profiles. 79
4.5 Empirical CDF of wire latency under the High-performance and Low-latency profile. 81

Chapter 1

Introduction

1.1 Background

Cryptography has always been an evolving and critical research field, as it directly affects core security concepts such as confidentiality, integrity, authentication, and non-repudiation. Modern systems, ranging from web applications to large-scale distributed infrastructures, rely heavily on cryptographic mechanisms to secure communication and protect sensitive information. Recently, quantum computing has brought cryptographic security back to the forefront, threatening the assumptions underlying widely deployed systems.

The emergence of quantum computers is expected to revolutionize computing by enabling efficient solutions to problems that classical computers cannot solve or struggle with [1]. However, it also poses high security risks, especially to public-key cryptography.

Security of the widely adopted public-key cryptography, such as RSA and elliptic-curve cryptography (ECC), primarily depends on complex mathematical problems like factorization and elliptic curve discrete logarithm. These problems will no longer be complex and secure with the emergence of quantum computing, due to Shor's algorithm, which uses quantum computation to solve such problems efficiently.

Responding to this challenge, a new branch of cryptography, known as post-quantum cryptography (PQC), has emerged, relying on mathematical problems believed to

remain secure against both classical and quantum attacks [1]. In August 2024, the U.S. National Institute of Standards and Technology (NIST) released the first finalized post-quantum cryptography standards, moving post-quantum cryptography from theoretical research to practical deployment [2]. This step was inevitable, driven by the "harvest now, decrypt later" (HNDL) attack model, in which adversaries collect and store valuable encrypted data, hoping to decrypt it once quantum computing becomes a reality [2].

However, deploying purely post-quantum key exchange mechanisms in existing systems faces many obstacles, including backward compatibility concerns, implementation immaturity, operational risk, and the need for extensive validation [3].

To address this problem, hybrid key-exchange mechanisms have been introduced, combining post-quantum algorithms with well-established classical ones, ensuring security against both classical and quantum attacks while supporting a gradual transition [4].

Modern deployments of secure communication protocols, including TLS 1.3, have already adopted a hybrid key exchange mechanism as part of the post-quantum transition efforts, reducing the associated risk [4]. However, a non-trivial trade-off between security and feasibility has been introduced, which becomes clearly noticeable in heterogeneous deployment environments, where endpoint and network capabilities differ. Current implementations fail to account for the operational context in which the channel is established, resulting in suboptimal performance or unnecessary overhead in certain scenarios.

This trade-off has resulted in various deployment strategies where some providers, like AWS and Cloudflare, offer hybrid post-quantum as an optional feature [5, 6], leaving many connections vulnerable to quantum attacks. On the other hand, systems with stricter security requirements enforce a fixed hybrid group with larger keys, risking operational failures in constrained environments [3].

Responding to this challenge, this work proposes AdaptiveQKE, a policy-driven hybrid post-quantum key exchange selection mechanism for TLS secure channels, based on a practical pre-handshake selection layer that dynamically determines the suitable hybrid group based on endpoint capabilities and network characteristics. The mechanism

is implemented as a client-server secure channel prototype that integrates hybrid key-exchange mechanisms using well-established cryptographic libraries, which achieve compliance with existing standards.
The effectiveness of the proposed mechanism is evaluated through quantitative analysis, comparing the performance of static configurations versus adaptive selection under various simulated network and device conditions. The expected outcome is an improved balance between security requirements and performance overhead, resulting in more efficient, context-aware post-quantum communication.

1.2 Problem Statement

The integration of hybrid key-exchange mechanisms in TLS 1.3, introduced to support post-quantum security, influences the computational and communication characteristics of the handshake process. By combining classical and post-quantum primitives, hybrid configurations may increase processing requirements and message sizes compared to traditional key exchange [7, 8].
In heterogeneous connectivity environments, where endpoint capabilities and network conditions vary across connections, the performance implications of hybrid key exchange are not uniform [9]. Different operational contexts may require different cryptographic configurations to achieve an appropriate balance between security guarantees and resource efficiency. However, TLS 1.3 does not provide a context-aware or policy-based mechanism for dynamically selecting among available hybrid key-exchange groups [7]. Instead, group negotiation relies on predefined mechanisms that do not incorporate explicit evaluation of device or network characteristics.
Consequently, the absence of an adaptive selection framework within TLS 1.3 limits the ability to align hybrid key-exchange configurations with the specific operational context of each connection. This gap between performance variability and static selection mechanisms constitutes the central problem addressed by this work.

1.3 Objectives

1.3.1 Design and Implement a Context-Aware Hybrid Group Selection Layer for TLS 1.3

To design and implement a policy-driven mechanism that selects among ML-KEM + ECDH hybrid groups according to measured round-trip time and endpoint resource limits, operating as a pre-handshake layer that requires no modification to the standardized TLS 1.3 handshake flow.

1.3.2 Evaluate Adaptive Selection Against Static Hybrid Configurations Across Device and Network Conditions

To quantitatively evaluate, under controlled experimental conditions, the performance impact of adaptive hybrid group selection in TLS 1.3 against static hybrid configurations, using TLS handshake size, CPU time and handshake latency as metrics, and to identify the conditions under which adaptive selection yields a measurable benefit.

1.4 Project Scope

Scope: This paper focuses on the design, implementation, and evaluation of AdaptiveQKE, a pre-handshake selection layer that adaptively selects hybrid key exchange groups (ML-KEM + ECDH) for TLS 1.3 secure channels based on measured round-trip time and endpoint resource limits. The scope includes the development of a prototype of a lightweight policy engine that maps contextual parameters to an appropriate hybrid group selected from a predefined allow-list. The proposed mechanism is implemented as a client-server layer using established and trusted cryptographic libraries to demonstrate feasibility. Additionally, the scope includes a performance evaluation conducted under controlled experimental conditions using quantitative metrics such as TLS handshake size, handshake latency, and CPU time to compare adaptive hybrid selection with static hybrid configurations. A comparative analysis across heteroge-

neous deployment scenarios is also performed to assess performance trade-offs and identify conditions under which adaptive selection provides measurable improvements in efficiency and resource utilization while maintaining compliance with TLS standards.

Out of Scope:

- The Design or modification of cryptographic algorithms.
- Any changes to the TLS protocol or its standardized handshake flow.
- Deployment in production environments or large-scale network infrastructures.
- Formal security proofs or exhaustive cryptanalytic analysis.
- Long-term operational evaluation or full infrastructure integration.

Although formal security analysis is outside the scope of this work, the implementation strictly adheres to existing cryptographic standards, recommendations, and best practices. The handshake flow itself is unmodified; the only departure from standardized identifiers is the use of a code point from the IANA private-use range for x25519_mlkem512, which has no assigned standard identifier and which is confined to the controlled experimental scope of this work.

1.5 Project Significance

This project proposes a policy-based pre-handshake selection layer that determines the suitable standardized hybrid key exchange group in TLS 1.3 according to security requirements and endpoint and network constraints, thereby reducing unnecessary CPU overhead, handshake latency, and handshake size caused by enforcing a single static hybrid configuration across heterogeneous environments. In addition to performance gains, the adaptive approach is motivated by two further considerations. Avoiding excessive computational load on constrained devices may reduce the physical leakage signals exploitable in side-channel attacks, and may limit the resource exhaustion that degrades service availability. Both are design motivations rather than evaluated outcomes, and neither is measured in this work. Enforcing computationally heavy hybrid groups on all connections can expose servers to denial-of-service risks and degrade the functionality of resource-limited clients such as IoT and mobile devices. This

selection preserves responsiveness and operational stability on both server and client sides. To address these challenges, this work introduces a prototype implementation of the proposed mechanism that balances performance, security, and availability in hybrid key exchange within secure channels, targeting system designers and security engineers integrating TLS-secured services. By comparing static and adaptive hybrid configurations, the study demonstrates how standardized hybrid groups can be selected more effectively under varying system and network conditions, supporting informed design decisions for future secure communication systems.

1.6 Report Structure

This report is organized into five chapters.

- **Chapter 1 - Introduction:** Introduces the project background, problem statement, objectives, scope, and significance of the study.
- **Chapter 2 - Literature Review:** Presents the theoretical background of TLS 1.3, quantum computing with post-quantum cryptography, and hybrid key exchange mechanisms. It also reviews related work in the field, followed by a gap analysis to identify the limitations of existing approaches and motivate the proposed solution.
- **Chapter 3 - Design & Methodology:** Presents the overall methodology of the proposed AdaptiveQKE framework, including the two-phase structure consisting of the design and implementation phase, followed by the evaluation and analysis phase. It also describes the project design which includes the architecture of the proposed AdaptiveQKE framework, hybrid key exchange construction, the policy engine and the experimental setup. Finally, it discusses the performance evaluation metrics.
- **Chapter 4 - Results & Analysis:** Presents the Results and analysis by comparing static and adaptive hybrid key exchange groups in TLS 1.3 in terms of CPU time, handshake latency, and handshake size.

- **Chapter 5 - Conclusions & Future Work:** Summarizes the project outcomes, states the boundaries within which the results hold, and highlights possible directions for future work.

Chapter 2

Literature Review

2.1 Introduction

This chapter presents the theoretical and research foundations that support the proposed AdaptiveQKE approach. It introduces the fundamental concepts related to secure communication in TLS 1.3, including key exchange mechanisms and post-quantum cryptography. In addition, hybrid key exchange techniques are introduced as part of the theoretical background for post-quantum secure communication. Furthermore, selected studies addressing post-quantum cryptography in secure communication protocols are reviewed, including works on performance evaluation, optimization, and adaptive mechanisms, with particular emphasis on performance implications under varying network and device conditions. Based on this review, a gap analysis is conducted to identify the limitations of existing approaches, especially the limited consideration of adaptive and context-aware mechanisms for selecting appropriate hybrid key exchange configurations within TLS. This analysis establishes the foundation for motivating the proposed AdaptiveQKE mechanism, which aims to enhance performance and efficiency in heterogeneous environments while maintaining strong security guarantees.

2.2 Theoretical Background

This section reviews the theoretical foundations underlying the proposed AdaptiveQKE mechanism, including TLS and its cryptographic foundations, the emerging quantum threat and post-quantum countermeasures, hybrid key exchange in TLS 1.3, and the principles of adaptive and context-aware selection.

2.2.1 TLS and Cryptographic Primitives

Transport Layer Security (TLS) is the main protocol used to secure communication over networks, making it the core environment in which the proposed system operates. Within TLS, the TLS handshake is the phase where the communicating parties establish the security settings of the connection, agree on the methods to be used, and prepare the session for secure data exchange. As part of this phase, cipher negotiation takes place, allowing the client and server to agree on the cryptographic algorithms or configurations that will be used for the session [7]. To support these functions, TLS relies on cryptographic primitives, which are the basic cryptographic building blocks used to construct secure protocols, such as key exchange and encryption mechanisms. Among these primitives, public-key cryptography plays a central role, since it enables the establishment of shared secrets and supports secure communication between parties over untrusted networks [7]. However, the security of these public-key mechanisms is directly threatened by advances in quantum computing, making the integrity of the TLS handshake a central concern in the post-quantum transition [1, 2].

2.2.2 Quantum Computing and Post-Quantum Cryptography

Quantum computing exploits quantum-mechanical phenomena, superposition and entanglement, to solve certain problems exponentially faster than classical computers [1]. Unlike classical bits, quantum bits (qubits) can exist in multiple states simultaneously, enabling quantum processors to explore large solution spaces in parallel. The cryptographic relevance of this was established by Shor's algorithm, which demonstrates that a sufficiently powerful quantum computer can solve both integer factorization and

discrete logarithm problems in polynomial time, which are the exact problems underpinning algorithms like RSA and ECC. This poses a critical threat to the public-key infrastructure securing modern TLS connections [1, 2]. The urgency of transitioning to quantum-resistant algorithms is further motivated by the harvest now, decrypt later (HNDL) attack model, in which adversaries collect and store encrypted traffic today with the intention of decrypting it once quantum hardware is available [2, 10]. In response, NIST standardized a set of post-quantum cryptography (PQC) algorithms, which are software-based schemes that can run on classical hardware, making them practical replacements for existing algorithms [2]. The primary algorithm relevant to this project is ML-KEM (Module-Lattice-Based Key Encapsulation Mechanism), derived from CRYSTALS-Kyber, offering increasing security across its three parameter sets at the cost of larger key and ciphertext sizes [11]. ML-KEM serves as the post-quantum component in the hybrid key exchange mechanisms discussed in the following section.

2.2.3 Hybrid Key Exchange in TLS 1.3

In TLS 1.3, classical key exchange mechanisms such as Elliptic Curve Diffie-Hellman (ECDH) are widely used to establish shared secrets between communicating parties [7, 8]. To address emerging quantum threats, hybrid key exchange mechanisms have been introduced, combining classical approaches with post-quantum techniques to enhance security [8]. These mechanisms are implemented through hybrid cryptographic groups, which define specific combinations of classical and post-quantum algorithms used during the handshake phase [8]. In cases where the selected configuration is not supported or is incompatible with device or network conditions, fallback mechanisms are employed to switch to alternative configurations [12]. While hybrid approaches improve security, they introduce performance overhead, including increased computational cost and handshake latency [13]. Furthermore, it is important to distinguish between traditional key exchange methods and post-quantum Key Encapsulation Mechanisms (KEMs), which follow a different approach for establishing shared secrets in hybrid TLS environments [8].

2.2.4 Adaptive and Context-Aware Selection

In heterogeneous environments, where devices possess varying capabilities and network conditions, applying a static hybrid key exchange configuration across all TLS connections regardless of device or network context leads to severe performance inefficiencies [14, 15]. A primary performance bottleneck in such deployments is handshake latency, which is dictated by the interaction between algorithm computational overhead and underlying network characteristics [9, 16]. To address these overheads, context-aware frameworks evaluate real-time environmental factors, such as bandwidth and endpoint processing constraints, to support informed configuration choices [17]. By continually processing these parameters, adaptive selection mechanisms dynamically adjust cryptographic parameters to optimize the trade-off between post-quantum security guarantees and system performance [17].

2.3 Related Works

The research on post-quantum TLS has progressed into two main directions: evaluating the performance impact of hybrid key exchange mechanisms and proposing solutions such as cryptographic modifications or adaptive approaches to cryptographic configuration. This section reviews the most relevant contributions and identifies the gap that *AdaptiveQKE* addresses.

Several studies have investigated the performance overhead introduced by integrating post-quantum key exchange into TLS 1.3. Souvatzidaki and Limniotis [13] compares pure post-quantum and hybrid (ML-KEM + ECDH) configurations under different network conditions, including varying packet loss ratios. Their results show that post-quantum approaches lead to additional communication and computational overhead, especially in constrained environments or when packet loss is high. However, hybrid configurations achieve performance close to classical methods, making them a practical option for transitioning to post-quantum security. Despite these findings, the study relies on static configurations and does not include a mechanism for dynamically selecting the most suitable key exchange scheme based on network or device conditions,

which reveals a gap in context-aware optimization.

On the implementation side, Zheng et al. [18] address the performance overhead of Post-Quantum TLS (PQ-TLS) by focusing on internal algorithm optimization. Instead of modifying protocol behavior or key exchange selection, they improve the implementation efficiency of ML-KEM through optimized operations such as polynomial multiplication and modular reduction using AVX-512, and integrate it into TLS 1.3. Their results show limited performance improvement in PQ-only mode, while in hybrid mode, the gain is minimal due to the dependence on both ML-KEM and ECDH. This indicates that optimizing a single algorithm does not significantly improve overall handshake performance. A key limitation is that the approach ignores the operational context of TLS connections, such as device and network conditions, making it complementary rather than a replacement for context-aware hybrid group selection.

Moving toward adaptive approaches, Zafar and Iqbal [12] propose a hybrid TLS framework that integrates code-based post-quantum cryptography, particularly the McEliece scheme, together with classical key exchange mechanisms. The framework employs a dual-handshake mechanism and uses HKDF to generate the session key. It also incorporates adaptive cipher negotiation and fallback mechanisms to select appropriate configurations based on device and network conditions, reverting to alternative schemes such as Kyber or classical TLS when necessary, and is implemented using OpenSSL with support from the Open Quantum Safe (OQS) project. However, the solution requires changes to the standard TLS protocol itself, instead of introducing a pre-handshake selection layer, which may lead to compatibility issues with existing systems and network components. Also, it does not address the dynamic selection among various lattice-based hybrid cryptographic groups, which may offer different trade-offs in terms of performance, key size, and security strength.

The closest work to our approach is that of Sengupta et al. [19], who propose a context-aware adaptive protocol (CAAP) that dynamically selects the most suitable PQC algorithm based on real-time operational conditions, without modifying the underlying cryptographic primitives. The proposed approach is evaluated within a 6G vehicular network environment, demonstrating measurable improvements in latency,

system stability, and resource efficiency while maintaining quantum-resistant security properties. However, the proposed solution operates at the application layer, which limits its applicability to specific deployment scenarios such as vehicular networks. This layer-dependent design restricts integration with widely adopted secure communication protocols that operate at lower layers, leaving the problem of adaptive cryptographic selection within the TLS handshake itself unaddressed.
Table 2.1 provides a structured comparison of all four reviewed works and the proposed framework across the key dimensions relevant to this project.

Table 2.1: Comparison of related works and the proposed AdaptiveQKE framework

Criterion	**Souvatzidaki et al. [13]**	**Zheng et al. [18]**	**Zafar & Iqbal [12]**	**Sengupta et al. [19]**	**Adaptive-QKE**
Primary Focus	Performance analysis	Algorithm optimization	Code-based hybrid	Adaptive selection	Adaptive hybrid selection
Key Exchange Type	Yes	Yes	Yes	No	Yes
Adaptive Selection	No	No	Partial	Yes	Yes
TLS Integration	Yes	Yes	Partial	No	Yes
Standards Compliance	Yes	Yes	Partial	N/A	Partial
Network-Aware	Yes	No	Yes	Yes	Yes
Device-Aware	No	No	Partial	Yes	Yes
Dynamic Group Selection	No	No	No	No	Yes
Pre-Handshake Layer	No	No	No	No	Yes

As shown in Table 2.1, none of the reviewed works provides a fully policy-driven, pre-handshake selection mechanism that dynamically chooses among standardized ML-KEM + ECDH hybrid groups within TLS 1.3 based on real-time endpoint and network conditions — in contrast to the proposed AdaptiveQKE mechanism shown in the final column, which directly addresses this gap.

2.4 Gap Analysis

The reviewed literature highlights significant efforts toward addressing the challenges of post-quantum cryptography deployment, including algorithm-level optimization, hybrid cryptographic integration, and performance evaluation of post-quantum key exchange mechanisms. These works collectively emphasize the importance of balancing security and performance during the transition to the post-quantum era, yet several critical gaps remain unaddressed.

Zheng et al. [18] demonstrate improvements in ML-KEM implementation efficiency through low-level algorithmic optimization. However, in hybrid mode, optimizing a single algorithm offers minimal handshake improvement, since performance is determined by both ML-KEM and ECDH. Additionally, the approach relies on a fixed configuration and ignores operational context, such as device capabilities or network conditions. This indicates that internal algorithm optimization alone is insufficient to address performance variability across heterogeneous environments.

Souvatzidaki and Limniotis [13] provide a thorough performance analysis of hybrid and pure post-quantum configurations under varying network conditions, establishing that hybrid configurations offer a practical balance between security and performance. However, their work relies entirely on static configurations and stops short of proposing a mechanism to dynamically select a suitable configuration based on observed conditions.

Zafar and Iqbal [12] take this a step further by introducing a hybrid TLS framework that includes adaptive cipher negotiation and fallback mechanisms. While this represents meaningful progress toward flexibility, the framework relies on a dual-handshake mechanism that extends the standard TLS flow, which may introduce compatibility and deployment challenges in existing infrastructures. In addition, the framework does not address the dynamic selection among NIST-standardized hybrid groups, limiting its ability to make fine-grained, standards-compliant decisions.

Sengupta et al. [19] present the closest conceptual alignment with the proposed work, demonstrating that adaptive cryptographic selection based on real-time operational context improves performance and stability without modifying underlying primitives.

However, their solution operates at the application layer and is tailored to a specific deployment domain, lacking the global applicability of a protocol-level solution.
Collectively, the reviewed works reveal a key limitation: existing approaches focus on algorithm-level optimization, static configurations, or limited forms of adaptability, without providing a fully policy-driven, context-aware solution. In particular, no prior work enables dynamic selection among ML-KEM + ECDH hybrid groups within the TLS 1.3 handshake based on real-time endpoint and network conditions. Adaptive-QKE addresses this gap by introducing a lightweight, policy-driven pre-handshake selection layer that bridges context awareness with protocol-level negotiation, offering a generalizable and standards-compliant solution applicable across heterogeneous deployment environments.

2.5 Summary

To conclude, this chapter introduced post-quantum cryptography and its applicability to TLS, and presented key theoretical concepts that are mainly related to this project. These concepts provided a clear understanding of how hybrid TLS and post-quantum cryptography operate in practice, followed by a review of recent research in this area. Some studies aimed at enhancing performance through internal optimization of PQ algorithms, while others explored hybrid frameworks or context-aware adaptive selection based on real-time conditions. Additionally, hybrid TLS frameworks have been suggested to increase the diversity in cryptography using code-based schemes, whereas performance evaluation studies highlighted the trade-offs between pure and hybrid configurations.
However, these works largely consider only particular aspects such as performance enhancement, reduced flexibility, or fixed hybrid configuration, and do not provide a mechanism for dynamically selecting between various hybrid TLS groups during the TLS handshake. Therefore, there remains a need for more flexible and context-aware approaches that support dynamic selection of hybrid TLS groups based on device capabilities, network conditions, and security requirements, which motivates the proposed work.

Chapter 3

Design & Methodology

3.1 Introduction

This chapter presents the design and methodology of AdaptiveQKE, a policy-driven pre-handshake selection mechanism for hybrid post-quantum key exchange in TLS 1.3, structured around two objectives: the design and implementation of a context-aware hybrid group selection layer for TLS 1.3, and the evaluation of adaptive selection against static hybrid configurations across device and network conditions. As established in Chapter 1, hybrid post-quantum key exchange introduces performance overhead that is not uniform across heterogeneous deployment environments, yet existing TLS 1.3 group negotiation does not account for these contextual factors. To address this gap, AdaptiveQKE introduces a lightweight pre-handshake layer that evaluates round-trip time and device capability, and selects the most appropriate hybrid group from a predefined allow-list before the TLS 1.3 handshake is initiated, preserving full protocol compliance. Section 3.2 presents the overall methodology and justification of key design choices. Section 3.3 describes the system architecture, hybrid key exchange construction, policy engine, and experimental setup. Section 3.4 defines the performance metrics, and Section 3.5 summarizes the chapter.

3.2 Overall Methodology

Figure 3.1 provides a high-level overview of the AdaptiveQKE methodology, organized into two phases aligned with the objectives of this work. In Phase 1, the AdaptiveQKE prototype is developed as a lightweight pre-handshake selection layer for hybrid key exchange in TLS 1.3. In Phase 2, the implemented mechanism is evaluated through comparison with static hybrid configurations under heterogeneous network and device conditions. The detailed description of these phases is presented in Sections 3.2.1 and 3.2.2, while the key design choices are justified in Section 3.2.3.

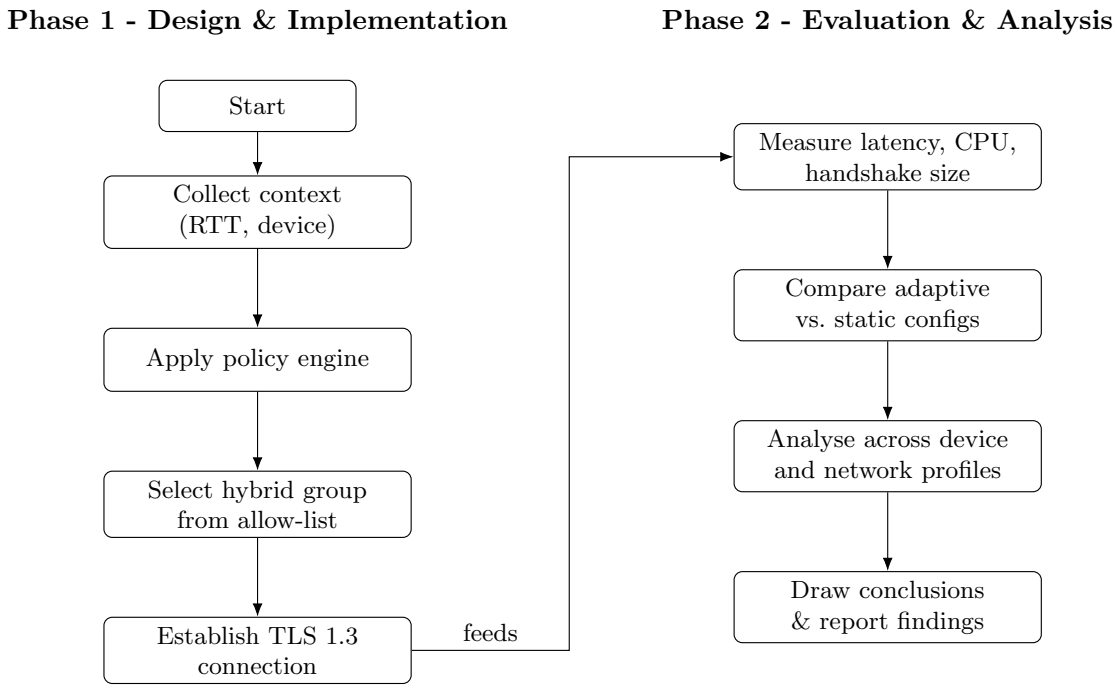


Figure 3.1: High-level overview of the AdaptiveQKE two-phase methodology

3.2.1 Phase 1 — Design and Implementation (Objective 1)

This phase focuses on implementing the AdaptiveQKE pre-handshake decision layer. Its purpose is to enable context-aware selection of a standardized hybrid key exchange group before the TLS 1.3 handshake begins. The implementation follows four sequential steps.

Step 1 - Context Collection

Before each adaptive TLS connection, the pre-handshake layer obtains a compact set of context indicators representing the network path and the endpoint resource constraints. The network-side context is represented by the round-trip time (RTT), which is estimated through three concurrent TCP connection attempts to a dedicated RTT probe service. Because the attempts are launched simultaneously, the first to complete successfully is retained as the RTT estimate; failed attempts are excluded so that a broken connection cannot be misread as a favorable RTT.
The endpoint-side context is represented quantitatively by the CPU share and memory limit associated with the device profile enforced for the current experimental condition through Linux cgroups v2. These configured resource values, together with the measured RTT, are supplied to the policy engine before the TLS client is configured. CPU and memory are considered relevant endpoint indicators because computational and memory constraints can affect the feasibility and performance of post-quantum TLS operations on resource-limited platforms [14].

Step 2 - Policy Evaluation

Once the context has been collected, it is passed to the policy engine, where the observed parameters are evaluated according to the selection logic defined in this work. At this stage, AdaptiveQKE performs a policy-driven assessment over a predefined allow-list of hybrid groups [4, 16] while remaining compatible with existing standards. The detailed decision logic of the policy engine is presented later in Section 3.3.3.

Step 3 - Hybrid Group Selection

Following policy evaluation, the process moves to hybrid group selection, where the selection layer determines the most suitable hybrid TLS group from the predefined allow-list according to the policy outcome produced in the previous step.

Step 4 - TLS Stack Configuration and Handshake Initiation

The selected hybrid group is then configured within the TLS implementation for use during the handshake. Once configured, the TLS 1.3 handshake is initiated using the chosen hybrid parameters, and the ClientHello message carries the selected group in the supported groups extension.
For example, the selected group can be configured in OpenSSL as follows [20]:

```
openssl s_client -groups X25519MLKEM768
```

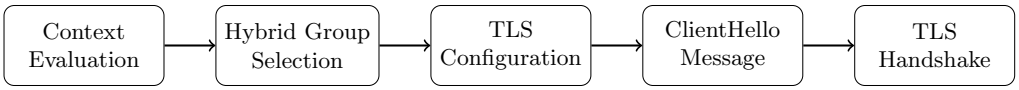


Figure 3.2: Phase 1 workflow of the adaptive hybrid TLS mechanism

3.2.2 Phase 2 — Evaluation and Analysis (Objective 2)

This phase addresses the quantitative evaluation of the adaptive selection approach followed in AdaptiveQKE against static hybrid configurations under controlled experimental conditions simulating heterogeneous device and network conditions. The evaluation follows a comparative approach across identical experimental conditions, with only the key exchange group varying between configurations. The evaluation is designed to answer two questions: first, whether the adaptive selection approach provided by AdaptiveQKE produces lower overhead than the static configuration of a high-security group in constrained environments; and second, whether it avoids defaulting to the lightest group when conditions are favorable enough to support a stronger one. The complete experimental design, including the definition of device profiles, network condition profiles, configurations under evaluation, and the experimental workflow, is presented in Section 3.3.4.

3.2.3 Justification of Design Choices

The design of AdaptiveQKE is shaped by four decisions that directly influence its reliability, performance, and deployability. Each decision is presented alongside its justification below.

Pre-handshake layer over protocol modification

Modifying the TLS handshake itself risks breaking interoperability and standards compliance, and would require changes to the TLS library that may introduce compatibility issues with existing systems and network components [12]. Therefore, AdaptiveQKE is implemented as a pre-handshake selection layer, ensuring full TLS 1.3 compliance without any protocol-level changes.

Threshold-based policy over machine learning

Applying machine learning before each TLS connection would be impractical: the overhead introduced far exceeds what the task demands, ML techniques are non-deterministic, and model training adds unnecessary management complexity. In contrast, the threshold-based policy introduces negligible computational overhead, typically in the sub-millisecond range, and is fully deterministic and auditable, ensuring consistency across conditions and straightforward reasoning for security decisions.

OQS-enabled OpenSSL as the implementation library

Building from scratch or using non-standard libraries would result in confounded performance results affected by the implementation rather than the configuration. As a result, the proposed system uses OQS-enabled OpenSSL, a widely adopted open-source library that provides production-grade, audited implementations of ML-KEM and hybrid TLS groups, ensuring that measured performance differences reflect algorithmic costs rather than implementation quality [13, 18].

Selection of hybrid groups

The three hybrid groups in the allow-list (`x25519_mlkem512`, `X25519MLKEM768`, and `SecP384r1MLKEM1024`) combine X25519 or secp384r1 as the classical component with the corresponding ML-KEM parameter set, spanning security levels 1, 3, and 5, respectively. `X25519MLKEM768` and `SecP384r1MLKEM1024` follow the IETF standardization draft [4], while `x25519_mlkem512` has no standardized code point and is instead identified by a value in the IANA private-use range, as implemented by the Open Quantum Safe provider [21]. It is included to cover the ML-KEM-512 security level within the controlled experimental scope of this work, consistent with prior post-quantum TLS evaluations that benchmark this hybrid at NIST security level 1 [16]. This selection spans the full range of NIST security levels, ensuring AdaptiveQKE has suitable choices across heterogeneous environments.

3.3 Project Design

3.3.1 System Architecture

The system architecture of the proposed AdaptiveQKE framework is illustrated in Figure 3.3. The architecture follows a client–server model, where the adaptive selection mechanism is implemented entirely on the client side as a pre-handshake layer, while the server operates using a standard TLS 1.3 implementation. On the client side, the system is composed of three main components: the context probe, the policy engine, and the TLS client. The context probe is responsible for collecting runtime information about the current connection environment, including round-trip time (RTT) and device capability indicators. RTT is obtained via a lightweight network probe (Section 3.3.3.1); device capability is represented by the CPU share and memory limit configured for the current device profile (Section 3.3.4.1), matching the limits enforced through Linux cgroups v2.

The collected context is passed to the policy engine, which evaluates the input parameters using a threshold-based decision logic and selects the most suitable hybrid key exchange group from a predefined allow-list. This selection is performed before

the handshake begins, forming a pre-handshake selection layer that does not modify the TLS protocol itself.

Once the hybrid group is selected, the TLS client incorporates the chosen configuration into the ClientHello message and initiates the TLS 1.3 handshake. On the server side, the TLS 1.3 server receives the ClientHello message, negotiates the selected hybrid group, and performs the key exchange using the underlying cryptographic libraries, including ML-KEM and ECDH.

The data flow in the system proceeds sequentially from context collection to policy evaluation and finally to handshake execution. This design ensures that the adaptive mechanism remains lightweight and fully compatible with standard TLS 1.3, as the handshake process itself is not modified and follows the standard protocol flow. In this implementation, RTT is obtained through a lightweight client-side probe to the target server.

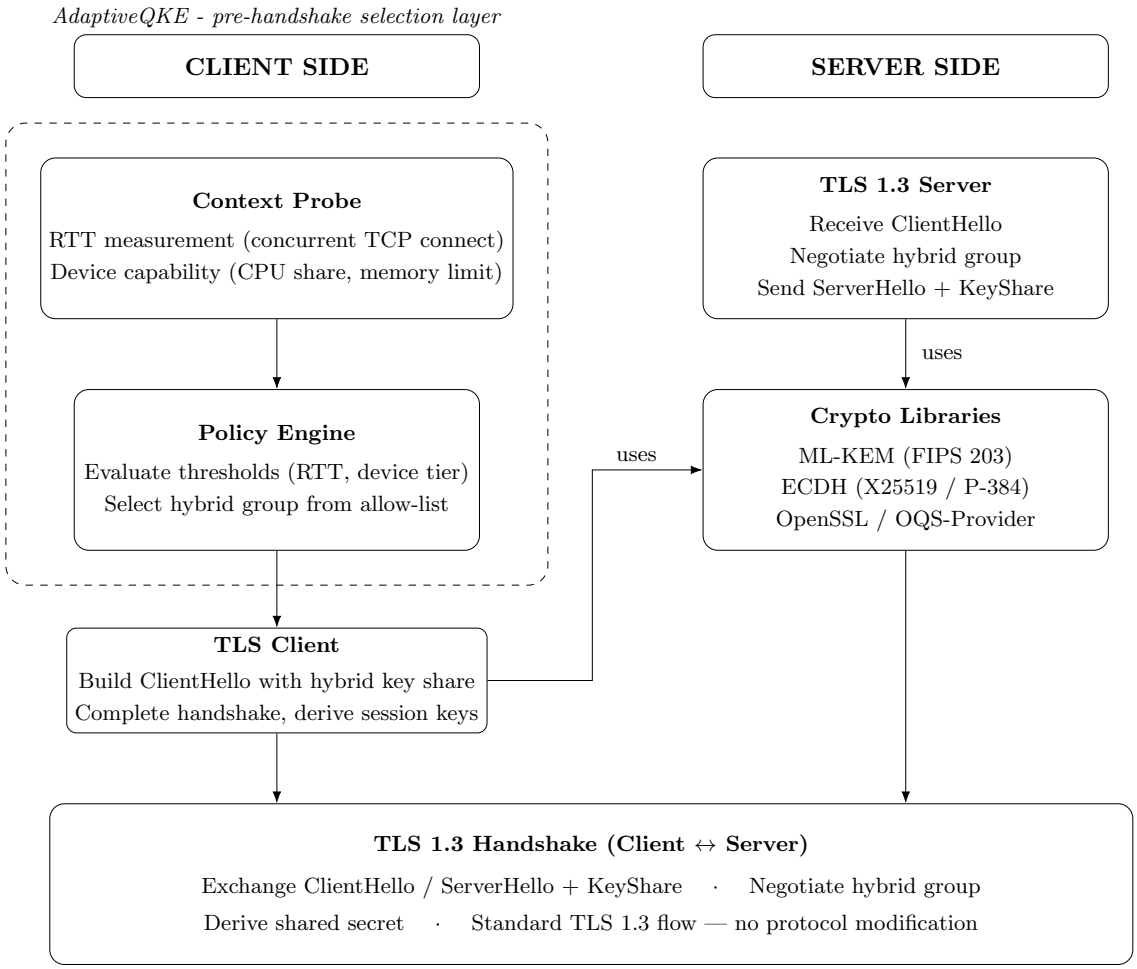


Figure 3.3: System architecture of the proposed AdaptiveQKE framework

3.3.2 Hybrid Key Exchange Construction

This section describes the hybrid key exchange construction, which assumes client and server support for hybrid TLS groups (Classical + ML-KEM) [12], and ensures that a secure session key is established.

Handshake Process:

1. **ClientHello** — The client initiates the TLS handshake by sending a ClientHello message containing the selected hybrid group in the supported groups extension,

along with classical and post-quantum key shares used to derive the shared secret between the client and the server.

2. **ServerHello** — The server checks whether it supports the proposed hybrid group. If supported, the server returns its own ephemeral classical public key pK_c and a post-quantum ciphertext ct, produced by encapsulating against the client's post-quantum public key pK_p; the handshake proceeds normally. If not supported, the server responds with a HelloRetryRequest and the system falls back to a compatible group [7].
3. **Hybrid Key Exchange** — Two shared secrets are generated independently: K_c (classical, e.g., ECDHE) and K_q (post-quantum, ML-KEM). Both parties compute the same values without transmitting them directly, as illustrated in Figure 3.4.

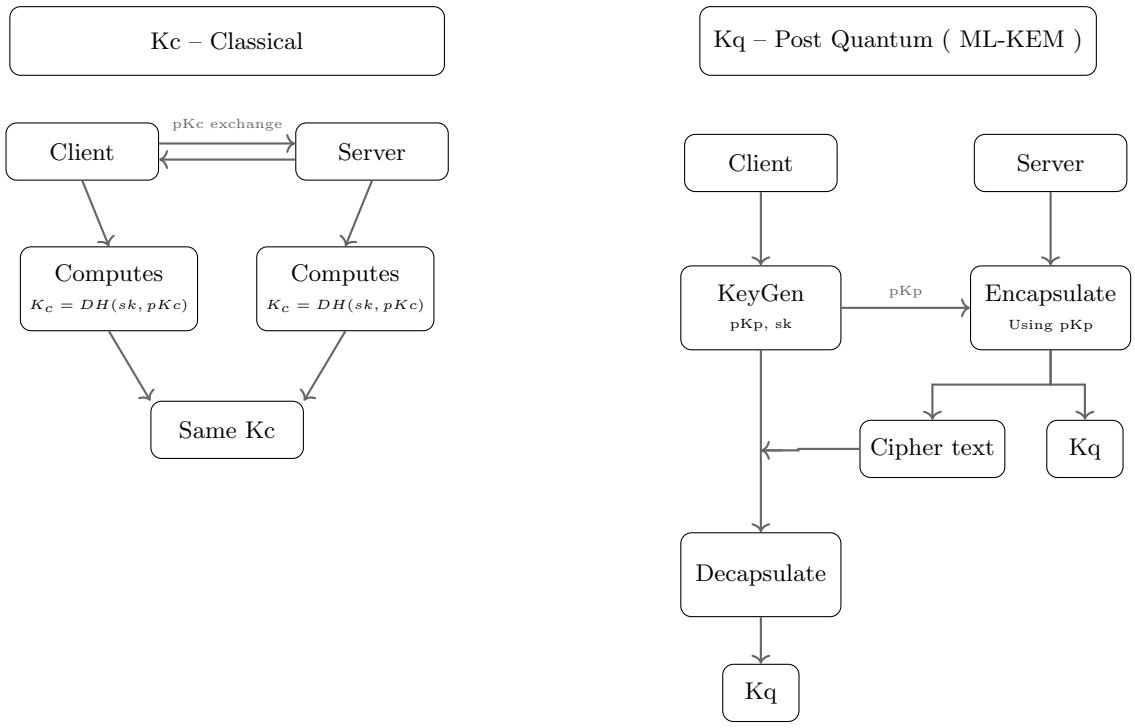


Figure 3.4: Classical and Post-Quantum Key Exchange Mechanisms

4. **Key Derivation** — The two secrets are combined using HKDF to produce

the final session key:

$$\text{Session Key} = \text{HKDF}(K_c \parallel K_q)$$

HKDF extracts a pseudorandom key from the combined secrets and expands it to generate the final session key. The order of concatenation is not fixed across hybrid groups; it is defined individually for each group by its governing specification.

5. **Session Establishment** — Both parties use the derived session key for encryption and authentication.
6. **Fallback Mechanism** — If the hybrid exchange is not supported, the system falls back to a compatible cryptographic group.

The complete hybrid TLS 1.3 handshake flow, including the fallback mechanism, is illustrated in Figure 3.5.

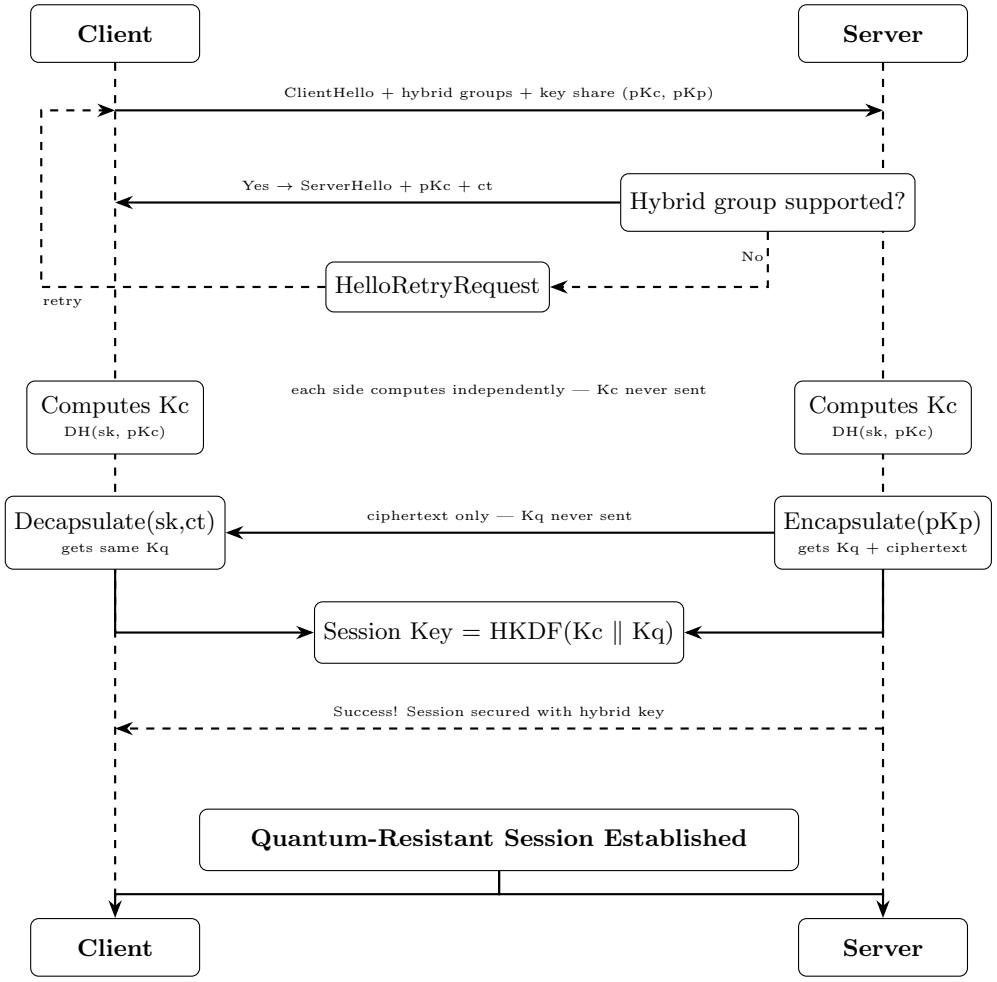


Figure 3.5: Hybrid TLS 1.3 Handshake Protocol Flow [7, 12]

3.3.2.1 Hybrid Group Parameters

Table 3.1 summarizes the three hybrid groups used in this work, covering all three NIST ML-KEM security levels along with their corresponding key and ciphertext sizes.

Table 3.1: Hybrid Key Exchange Parameters, all sizes in bytes ([4, 11, 16, 21–24])

Hybrid Group	Classical	PQ Algorithm	Security Level	Client Share	Server CT[a]
`x25519_mlkem512`	X25519	ML-KEM-512	1	832	768
`X25519MLKEM768`	X25519	ML-KEM-768	3	1216	1088
`SecP384r1MLKEM1024`	secp384r1	ML-KEM-1024	5	1665	1568

[a]ML-KEM ciphertext only; excludes the server's classical ephemeral share (+32 B for X25519, +97 B for secp384r1).

3.3.3 Policy Engine

The policy engine is the core decision-making component of the AdaptiveQKE pre-handshake selection layer. Its role is to evaluate the collected network and endpoint context and select a context-appropriate hybrid key exchange group before the TLS 1.3 handshake begins. The implemented policy follows a staged, threshold-based decision process in which the measured round-trip time is first used to determine a candidate group. Endpoint resource constraints, represented by the available CPU share and memory limit, are then used to classify the client into a device tier and validate whether the candidate is suitable for that tier. A conceptual extension to this design, applying application-declared bounds on the selected security level, is discussed at the end of this section.

This separation allows connection-dependent network conditions and endpoint resource limitations to influence the final configuration without modifying the standardized TLS 1.3 handshake flow. Once the final group has been selected, it is configured in the TLS client before the handshake is initiated [4, 7].

3.3.3.1 Inputs Description

The policy engine operates on three quantitative inputs: round-trip time (RTT), allocated CPU share, and memory limit. RTT, measured in milliseconds, represents the delay of the current network path and is obtained during the pre-handshake

context collection phase. CPU share, expressed as a percentage of one core, and memory limit, expressed in megabytes, represent the endpoint resource constraints enforced through Linux cgroups v2.
RTT is used to determine the initial candidate group, since network delay directly affects TLS 1.3 handshake latency [7]. CPU share and memory limit are then used to classify the endpoint and validate whether the candidate is suitable for its available resources [14]. When the two resource indicators correspond to different tiers, the more restrictive tier is applied.

3.3.3.2 General Decision Logic

The policy engine follows a two-stage decision strategy. First, it derives a candidate group based on the measured round-trip time (RTT). Second, it validates the candidate against the client device tier, which is classified using the available CPU share and memory limit. This validation may preserve the original candidate or restrict it to a lower-cost hybrid group according to the configured device-capability rules. The resulting group is then supplied to the TLS client before connection establishment. Therefore, the adaptive selection is performed as a pre-handshake decision layer without modifying the standard TLS 1.3 handshake flow [7].

RTT-Driven Candidate Determination

The decision process begins by assigning an initial candidate group according to the measured round-trip time (RTT). RTT is used as the network-side policy input because it varies across connections and contributes directly to the latency of the TLS 1.3 handshake exchanges [7, 13]. Network rate and packet loss are not used as direct policy inputs; they remain controlled environmental characteristics within the experimental network profiles.
The measured RTT is divided into three operational regions. When RTT exceeds the upper threshold (T_{hi}), the connection is treated as latency-constrained, and the lightest hybrid group, x25519_mlkem512, is assigned as a conservative candidate. When RTT falls below the lower threshold (T_{lo}), the connection is treated as latency-favorable, and

SecP384r1MLKEM1024 is assigned as the strongest candidate. RTT values between the two thresholds are treated as moderate, and X25519MLKEM768 is selected as the balanced candidate.

This threshold-based mapping provides deterministic and complete coverage of the RTT space. Values equal to either threshold fall within the moderate region. The candidate produced at this stage is not necessarily the final group, since it is subsequently validated against the endpoint device tier.

Table 3.2: RTT-Driven Candidate Determination

RTT Condition	**Candidate Group**
RTT $> T_{hi}$	x25519_mlkem512
RTT $< T_{lo}$	SecP384r1MLKEM1024
$T_{lo} \leq$ RTT $\leq T_{hi}$	X25519MLKEM768

Device-Aware Candidate Validation

After the RTT-driven candidate has been determined, the policy engine classifies the endpoint according to its allocated CPU share and memory limit. When the two indicators correspond to different capability tiers, the more restrictive tier is applied. The resulting device tier is then used to validate or cap the candidate according to the resource constraints of the client.

For the **high-performance profile**, no additional restriction is applied, and the RTT-selected candidate is preserved.

For the **mid-range profile**, the strongest candidate is capped at X25519MLKEM768, while the light and balanced candidates are retained.

For the **constrained profile**, the final selection is conservatively restricted to x25519_mlkem512 to limit computational demand under severe CPU and memory constraints [14].

Table 3.3: Device-Aware Candidate Validation

Candidate	High-performance	Mid-range	Constrained
512	512	512	512
768	768	768	512
1024	1024	768	512

These mappings represent conservative prototype policy constraints rather than universal hardware feasibility boundaries. They do not imply that heavier groups cannot execute on limited devices; instead, they define the resource-aware selection behavior evaluated in this work.

3.3.3.3 Formal Selection Equations

The selection process is formalized in two main stages. First, the policy engine derives an initial candidate group from the measured RTT. Second, the endpoint is classified according to its allocated CPU share and memory limit, and the candidate is validated against the resulting device tier to determine the final selected group.
Let the three policy tiers be defined as follows:

$$G_{512} = \texttt{x25519_mlkem512}, \quad G_{768} = \texttt{X25519MLKEM768}, \quad G_{1024} = \texttt{SecP384r1MLKEM1024},$$
$$G_{512} < G_{768} < G_{1024},$$

where the ordering represents the increasing group tier used by the policy engine.
The RTT-driven candidate group is defined as:

$$G_{\text{RTT}} = \begin{cases} G_{512}, & \text{if } RTT > T_{\text{hi}}, \\ G_{1024}, & \text{if } RTT < T_{\text{lo}}, \\ G_{768}, & \text{if } T_{\text{lo}} \leq RTT \leq T_{\text{hi}}. \end{cases} \tag{3.1}$$

Let C denote the allocated CPU share in percent and M denote the memory limit in megabytes. Unrestricted resources are treated as unbounded values. The endpoint device tier is classified as:

$$D = \begin{cases} \text{Constrained}, & \text{if } C < C_{\text{con}} \vee M < M_{\text{con}}, \\ \text{Mid-range}, & \text{if } C \geq C_{\text{con}} \wedge M \geq M_{\text{con}} \wedge (C < C_{\text{mid}} \vee M < M_{\text{mid}}), \\ \text{High-performance}, & \text{if } C \geq C_{\text{mid}} \wedge M \geq M_{\text{mid}}. \end{cases} \tag{3.2}$$

The constrained condition is evaluated first so that, when CPU and memory correspond to different tiers, the more restrictive classification is applied. The final selected group is then defined as:

$$G^* = \begin{cases} G_{\text{RTT}}, & \text{if } D = \text{High-performance}, \\ \min(G_{\text{RTT}}, G_{768}), & \text{if } D = \text{Mid-range}, \\ G_{512}, & \text{if } D = \text{Constrained}. \end{cases} \tag{3.3}$$

The resulting rule-based procedure is summarized in Algorithm 1.

3.3.3.4 Policy Engine Pseudocode

Algorithm 1 Pseudocode of the Policy Engine

Require: RTT ▷ round-trip time in ms
Require: C ▷ allocated CPU share in percent
Require: M ▷ memory limit in MB
Require: $T_{\text{lo}}, T_{\text{hi}}$ ▷ lower and upper RTT thresholds
Require: $C_{\text{con}}, C_{\text{mid}}$ ▷ CPU classification thresholds
Require: $M_{\text{con}}, M_{\text{mid}}$ ▷ memory classification thresholds
Ensure: G^* ▷ final selected hybrid group

Stage 1: RTT-Driven Candidate Determination
if $RTT > T_{\text{hi}}$ **then**
    $G_{\text{RTT}} \leftarrow$ `x25519_mlkem512`
else if $RTT < T_{\text{lo}}$ **then**
    $G_{\text{RTT}} \leftarrow$ `SecP384r1MLKEM1024`
else
    $G_{\text{RTT}} \leftarrow$ `X25519MLKEM768`
end if

Stage 2: Device Classification and Candidate Validation
if $C < C_{\text{con}}$ **or** $M < M_{\text{con}}$ **then**
    $D \leftarrow$ Constrained
else if $C < C_{\text{mid}}$ **or** $M < M_{\text{mid}}$ **then**
    $D \leftarrow$ Mid-range
else
    $D \leftarrow$ High-performance
end if

if $D =$ High-performance **then**
    $G^* \leftarrow G_{\text{RTT}}$
else if $D =$ Mid-range **then**
    if $G_{\text{RTT}} =$ `SecP384r1MLKEM1024` **then**
        $G^* \leftarrow$ `X25519MLKEM768`
    else
        $G^* \leftarrow G_{\text{RTT}}$
    end if
else ▷ $D =$ Constrained
    $G^* \leftarrow$ `x25519_mlkem512`
end if
return G^*

3.3.3.5 Decision Flowchart

Figure 3.6 summarizes the same two-stage decision process visually: RTT-driven candidate determination followed by device-aware candidate validation.

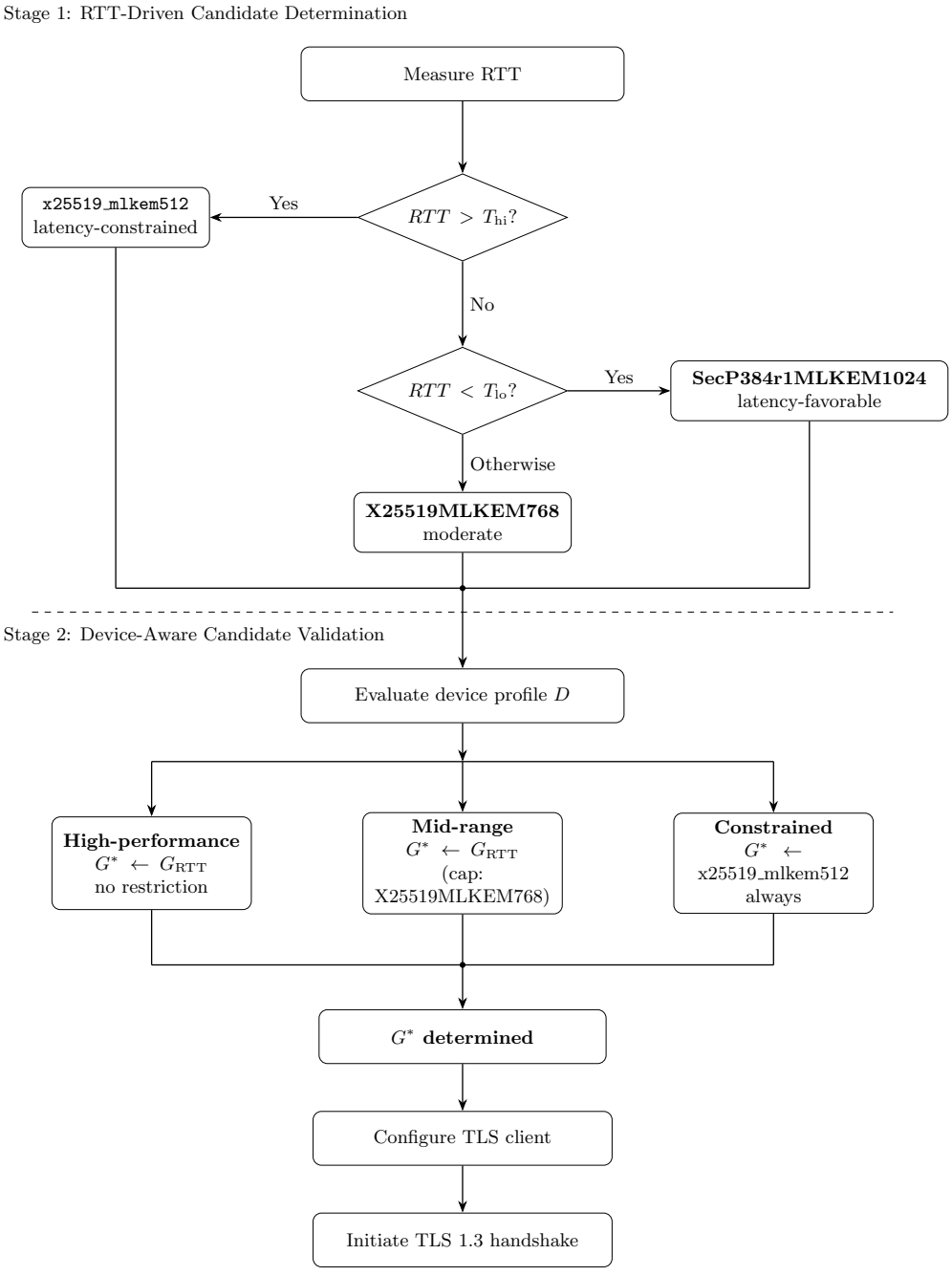


Figure 3.6: AdaptiveQKE policy engine decision flowchart

3.3.3.6 Threshold Rationale

The thresholds used by the policy engine are fixed prototype design parameters rather than universal or empirically optimal performance boundaries. Two sets of thresholds are defined. The RTT thresholds divide the measured network delay into latency-favorable, moderate, and latency-constrained regions, while the CPU and memory thresholds classify the endpoint into high-performance, mid-range, or constrained resource tiers. The selected values are aligned with the controlled network and device profiles used in the experimental evaluation.

Rationale for RTT Thresholds

The RTT thresholds are set to $T_{\text{lo}} = 50$ ms and $T_{\text{hi}} = 100$ ms. These values are motivated by RTT ranges used in previous post-quantum TLS performance evaluations and provide practical boundaries for distinguishing the network conditions considered in this work [13].

An RTT below T_{lo} is treated as latency-favorable, whereas an RTT above T_{hi} is treated as latency-constrained. Values equal to or between the two thresholds are assigned to the moderate region. These thresholds define the RTT regions used by the prototype to select the initial candidate group; they are not intended as universal network performance boundaries.

Rationale for Device Classification Thresholds

Endpoint classification is based on the allocated CPU share and memory limit. A CPU share below $C_{\text{con}} = 15\%$ or a memory limit below $M_{\text{con}} = 64$ MB classifies the endpoint as constrained. These lower boundaries lie between the enforced constrained profile, configured with 10% CPU and 32 MB of memory, and the mid-range profile, configured with 25% CPU and 128 MB of memory.

The upper classification thresholds are set to $C_{\text{mid}} = 60\%$ and $M_{\text{mid}} = 256$ MB. An endpoint that is not classified as constrained but falls below either upper threshold is classified as mid-range. An endpoint meeting or exceeding both upper thresholds is classified as high-performance. When CPU share and memory limit correspond

to different tiers, the more restrictive classification is applied. The relevance of these resource constraints to post-quantum TLS performance on limited platforms is supported by prior work [14]; however, the exact threshold values are prototype parameters selected to distinguish the resource profiles evaluated in this study.

Table 3.4: Policy Engine Threshold Values and Rationale

Parameter	**Value**	**Role**	**Basis**
T_{lo}	50 ms	Lower RTT boundary; values below it are treated as latency-favorable	Selected from RTT ranges used in prior post-quantum TLS evaluations and to distinguish the low-latency experimental region [13]
T_{hi}	100 ms	Upper RTT boundary; values above it are treated as latency-constrained	Selected to separate the high-latency experimental region from the moderate region [13]
C_{con}	15%	CPU share below which the endpoint is classified as constrained	Placed between the enforced constrained profile (10%) and mid-range profile (25%)
C_{mid}	60%	CPU share below which a non-constrained endpoint is classified as mid-range	Prototype upper boundary separating explicitly limited CPU profiles from the unrestricted profile
M_{con}	64 MB	Memory limit below which the endpoint is classified as constrained	Placed between the enforced constrained profile (32 MB) and mid-range profile (128 MB)
M_{mid}	256 MB	Memory limit below which a non-constrained endpoint is classified as mid-range	Prototype upper boundary separating explicitly limited memory profiles from the unrestricted profile

3.3.3.7 Security Level Bounds (Stage 0, Conceptual)

In addition to the two implemented stages, the policy engine's design includes a conceptual Stage 0: an application-declared pair of bounds on the selected security level, independent of network or device conditions. This stage is presented here as part of the design rationale; it is not implemented or evaluated in this work, and no trial in the experimental evaluation exercises it.

Two distinct motivations justify the bounds. The lower bound, or security floor, answers a downgrade-attack concern that Stages 1 and 2 do not address on their own: because RTT-driven candidate determination (Stage 1) responds to a measured, and therefore potentially manipulable, network signal, an adversary capable of influencing the measured RTT (for example, by inducing artificial latency) could attempt to force the policy toward a weaker hybrid group than the deploying application requires. The upper bound, or security ceiling, addresses a different concern: even where device and network conditions would support the strongest hybrid group, a deploying application may prefer to reserve the associated computational cost for other workloads rather than spend it by default whenever conditions allow it. This differs from Stage 2's device-tier capping, which reflects what the endpoint can feasibly run; the Stage 0 ceiling reflects what the application chooses to spend, independent of feasibility.

The application therefore declares a minimum acceptable security level $S_{\min}$ and a maximum acceptable security level $S_{\max}$, with $S_{\min} \leq S_{\max}$, both drawn from $\{1, 3, 5\}$ corresponding to the NIST security levels of the three allow-listed groups. The final selected group clamps the output of Stage 2 within these bounds:

$$G^*_{\text{final}} = \max(G_{S_{\min}}, \min(G^*, G_{S_{\max}})) \tag{3.4}$$

where $G_{S_{\min}}$ and $G_{S_{\max}}$ are the hybrid groups corresponding to the declared bounds, and G^* is the output of Stage 2 (Equation 3.3). Because the floor can only raise the selected group and the ceiling can only lower it, Stage 0 can never be exploited to force a downgrade below the application's declared minimum, and can never exceed the application's declared maximum, regardless of what an attacker manipulates in the RTT measurement or how favorable the measured conditions are.

No application-declared bounds are used in the experimental evaluation presented in Chapter 4. The selection is therefore equivalent to the widest admissible pair ($S_{\min} = 1$, $S_{\max} = 5$), for which $G_{S_{\min}} \leq G^* \leq G_{S_{\max}}$ holds for any output of Stage 2 and the clamp in Equation 3.4 is inert by construction; the prototype consequently omits the clamp rather than evaluating it with permissive bounds. This isolates the performance evaluation to the adaptive mechanism itself; Stage 0's protective and resource-budgeting behavior is a property of the design rather than something exercised or measured by the experiments in this chapter.

3.3.4 Experimental Setup

This section defines the experimental setup used to implement and evaluate Adaptive-QKE across heterogeneous deployment environments.

3.3.4.1 Device Profiles

To simulate the heterogeneity of real-world client-side environments, AdaptiveQKE defines three device profiles based on the computational resources available to the client process, specifically CPU capacity and memory. The enforcement of resource constraints is applied programmatically using Linux cgroups (control groups) [25], allowing precise, repeatable limiting of CPU time and memory allocation without requiring dedicated physical hardware.CPU quotas are enforced using a shortened cgroup v2 CFS period of 10 ms rather than the 100 ms default, while preserving the same CPU fraction. Under the default period, exhausting the quota mid-handshake would freeze the client process for up to the remaining 75–90 ms of that period, producing an artificial bimodal latency tail that would dominate the handshake latency being measured; the shortened period caps this stall at roughly 7–9 ms. While mid-range and constrained hardware targets often have less memory than what is specified, the limits are selected to account for the mandatory overhead of the Linux runtime environment when enforced via cgroups [25], ensuring that performance degradation is a result of cryptographic load rather than OS-level instability. The three profiles are described below and summarized in Table 3.5.

3.3.4.1.1 High-performance profile: This profile represents a desktop workstation, cloud server, or high-end laptop, and places no restrictions on CPU or memory. It serves as the baseline against which the overhead of hybrid post-quantum key exchange is measured. The performance difference between hybrid groups in this profile is expected to be driven primarily by computational cost differences between ML-KEM parameter sets.

3.3.4.1.2 Mid-range profile: This profile represents a mid-range smartphone or tablet, with the client process limited to 25% of a single CPU core and 128 MB of RAM. At this level of constraint, the additional computation required by higher ML-KEM parameter sets becomes more noticeable, and the policy engine's selection of a lighter group is expected produce measurable latency improvements.

3.3.4.1.3 Constrained profile: This profile represents a resource-constrained IoT device or embedded sensor, with the client process limited to 10% of a single CPU core and 32 MB of RAM. Post-quantum handshake studies have shown that on severely constrained devices, handshake overhead can be 1.6 to 10.6 times greater than classical TLS, depending on the security level [14].

Table 3.5: Simulated device profiles

Profile	CPU Limit	RAM Limit	Device Class
High-performance	Unrestricted	Unrestricted	Desktop / cloud server
Mid-range	25% of one core	128 MB	Mid-range smartphone / tablet
Constrained	10% of one core	32 MB	IoT sensor / embedded device

3.3.4.2 Network Condition Profiles

Three network condition profiles are defined to model the range of connectivity environments encountered in real-world TLS deployments. The profiles differ across three network parameters: round-trip time (RTT), available bandwidth (BW), and packet loss probability. While bandwidth and packet loss are not direct inputs to the policy engine, they are included as environmental characteristics of each profile to

accurately reflect real-world connectivity conditions and their influence on handshake performance across configurations. Network conditions are simulated using tc-netem (Traffic Control Network Emulator) [26], a Linux kernel module that allows configuring delay, bandwidth, and packet loss on a network interface with high precision and repeatability, ensuring that the profiles are applied consistently across all experimental conditions. The role of RTT in determining handshake performance is detailed in Section 3.3.3.1. The loopback interface's MTU is additionally forced down from its default of 65,536 bytes to the standard Ethernet value of 1,500 bytes for the duration of each profile, and restored afterward. Without this, handshake traffic would not be fragmented into multiple packets on the loopback path, and neither the per-packet serialization delay applied through netem's rate option nor the congestion-window argument in Section 3.4.3.2 would apply as they would on a real network. The three profiles are described below and summarized in Table 3.6.

3.3.4.2.1 Low-latency profile: This profile models a local area network or high-quality broadband connection, with RTT below T_{lo} (50 ms), bandwidth above B_{hi} (20 Mbps), and zero packet loss. Under these conditions, the performance overhead introduced by all three hybrid groups is expected to be minimal.

3.3.4.2.2 Moderate profile: This profile models a typical broadband or mobile connection, with RTT between T_{lo} and T_{hi} (50–100 ms), bandwidth between B_{lo} and B_{hi} (5–20 Mbps), and a packet loss probability of 0.5%. It represents the most common real-world scenario for TLS connections. Under these conditions, the overhead of heavier hybrid groups becomes more noticeable.

3.3.4.2.3 High-latency profile: This profile models a constrained wireless or cellular connection, with RTT above T_{hi} (100 ms), bandwidth below B_{lo} (5 Mbps), and a packet loss probability of 2%. At high packet loss rates, retransmissions of large key material disproportionately affect post-quantum handshakes [13], making this profile the most sensitive to group selection.

Table 3.6: Simulated network condition profiles and parameters

Profile	**RTT**	**Bandwidth**	**Packet Loss**	**Configured (tc-netem)**
Low-latency	< 50 ms	> 20 Mbps	0%	delay=10ms, rate=100Mbit, loss=0%
Moderate	50–100 ms	5–20 Mbps	0.5%	delay=35ms, rate=15Mbit, loss=0.5%
High-latency	> 100 ms	< 5 Mbps	2%	delay=60ms, rate=3Mbit, loss=2%

The RTT, bandwidth, and packet-loss columns describe the real-world connectivity scenario each profile models, expressed as ranges; the Configured column lists the exact tc-netem parameters applied to produce that scenario, from which the modeled ranges follow but which the delivered path may deviate from slightly due to kernel timer granularity and scheduling.

3.3.4.3 Configurations Under Evaluation

Four configurations are evaluated for each combination of device and network condition profiles, representing the full comparison space between static and adaptive group selection. Three of the four are static baselines and one is the proposed adaptive mechanism.

Static-512 (`x25519_mlkem512` applied uniformly): This configuration forces the lightest hybrid group for all connections regardless of device or network context, providing only NIST security level 1. It represents the performance-priority deployment strategy, minimizing handshake overhead unconditionally at the cost of reduced post-quantum security strength.

Static-768 (`X25519MLKEM768` applied uniformly): This configuration forces the balanced hybrid group for all connections, providing NIST security level 3. It represents the middle-ground baseline and is expected to perform adequately across most conditions, but suboptimally at the extremes.

Static-1024 (`SecP384r1MLKEM1024` applied uniformly): This configuration forces the strongest hybrid group for all connections, providing NIST security level 5. It represents the security-maximizing deployment strategy, imposing the highest computational

and communication overhead unconditionally, regardless of whether conditions would support a lighter alternative.

Adaptive (AdaptiveQKE; policy engine selects per connection): The hybrid group is selected dynamically by the policy engine based on the measured RTT and endpoint device capability (CPU share and memory limit) prior to each handshake, as described in Section 3.3.3. This configuration is the proposed mechanism and is expected to outperform static configurations by selecting a lighter group under constrained conditions and a stronger group when conditions allow, achieving a better balance between security and performance across the experimental conditions.

3.3.4.4 Experimental Environment

The prototype is implemented and evaluated on a machine running Ubuntu 22.04 LTS, selected for its stable support of cgroups v2 and tc-netem, and its widespread use as the standard environment in post-quantum TLS evaluation studies [13]. The TLS client and server are built on OQS-enabled OpenSSL, an established choice for post-quantum TLS evaluation [13] that provides production-grade implementations of ML-KEM and the hybrid groups required for this work. The specific build used is OQS provider 0.9.0 (848b4e6) aligned with liboqs 0.13.0, captured directly from the experimental environment (Table 3.7). The client and server communicate over a loopback interface on the same machine, eliminating external network variability and ensuring that all observed performance differences are attributable solely to the configuration under evaluation. TCP congestion control is fixed to BBR for the duration of each run, together with the kernel's default queueing discipline set to fq (fair queueing), which BBR requires to pace packet transmission accurately; without it, BBR's pacing has no effect. The original congestion-control algorithm is restored after each run. The default queueing discipline is not restored, though this has no bearing on the measurements themselves, since tc-netem attaches its own explicit qdisc to the loopback interface for every measured condition, overriding the default in all cases that matter. The automation script, cgroups enforcement, tc-netem configuration, and performance measurement are implemented in Python 3.10.12, with getrusage (Python's `resource` module) used for per-process CPU time measurement and a custom byte-counting

TCP relay used for handshake size and latency capture, as detailed in Section 3.4.3. The complete experimental workflow is automated, as described in Section 3.3.4.5. Table 3.7 summarizes the experimental environment specifications.

Table 3.7: Experimental environment specifications

Component	**Specification**
Operating System	Ubuntu 22.04 LTS
Kernel	Linux 6.8.0-124-generic
OpenSSL	OpenSSL 3.4.1
OQS Provider	oqs-provider 0.9.0 (848b4e6)
liboqs	0.13.0
Programming Language	Python 3.10.12
Network Interface	Loopback (localhost)
Congestion Control	BBR
CPU Measurement	getrusage (Python `resource` module)
Byte Accounting	Custom byte-counting TCP relay
Network Emulation	tc-netem (iproute2 5.15.0)
Resource Limiting	Linux cgroups v2
Loopback MTU	1500 B (forced; kernel default 65536 B)
CFS Period	10 ms (cgroup v2 default: 100 ms)
CPU	Intel Core i5-1035G1 @ 1.00 GHz
Cores/vCPUs	2 (2 cores, 1 thread/core)
Virtualization	VMware, full virtualization

3.3.4.5 Experimental Workflow

The experimental workflow follows a nested loop structure, as illustrated in Figure 3.7. The outer loop iterates over the three device profiles, applying the corresponding cgroups limits before any trials begin; the middle loop iterates over the three network profiles within each device profile, applying the corresponding tc-netem parameters. For each resulting device-network scenario, all four configurations and their N trials

are combined into a single execution order and interleaved via a seeded shuffle (seed = 20260719), rather than run as sequential per-configuration blocks; this prevents systematic drift over the course of a scenario's run (e.g. thermal effects, cgroup warm-up) from confounding one configuration more than another. Each trial then executes a single TLS handshake and records the three performance metrics defined in Section 3.4.

The total number of experimental conditions is given by:

$$C_{total} = N_{device} \times N_{network} \times N_{config} \tag{3.5}$$

where $N_{device} = 3$, $N_{network} = 3$, and $N_{config} = 4$, resulting in 36 distinct experimental conditions. Each condition is repeated over $N = 50$ trials, consistent with the trial count adopted in prior post-quantum TLS evaluation studies [13], producing a total of

$$M_{total} = C_{total} \times N = 1{,}800 \tag{3.6}$$

handshake measurements. Raw per-trial measurements are retained; aggregation into medians, interquartile ranges, and bootstrap 95% confidence intervals is performed during analysis, not during data collection. This trial count provides stable estimates without imposing excessive experiment runtime.

To ensure measurement integrity, TLS session tickets are disabled on both client and server ($-no_ticket$), and each trial launches a fresh client process, so no session state can persist between trials and every measured handshake is a full handshake. Each trial captures a complete TLS 1.3 handshake from ClientHello to Finished with no application data exchanged, isolating handshake performance from data transfer effects.

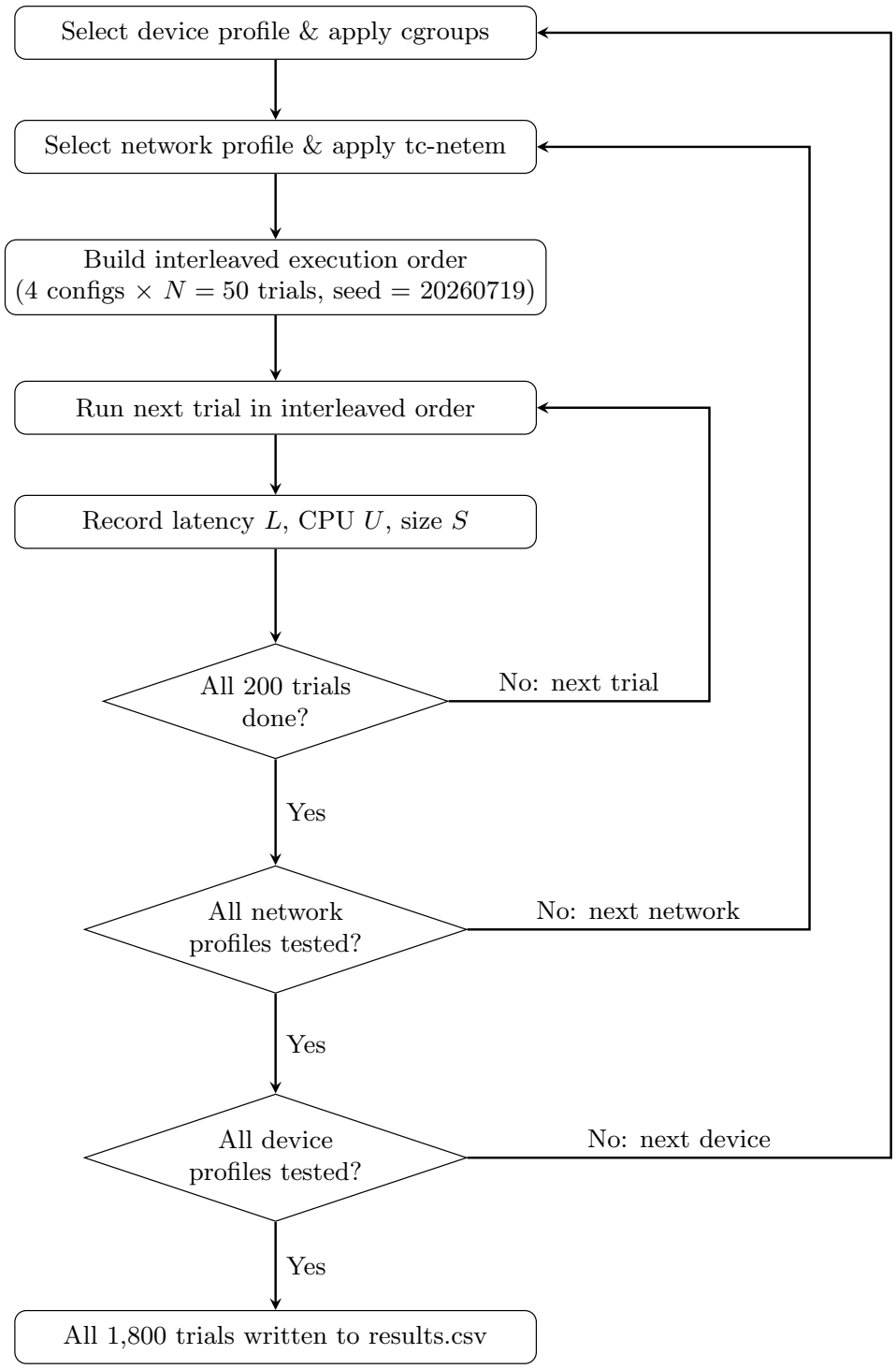


Figure 3.7: Experimental workflow across profile combinations and configurations

3.3.4.6 Crypto-Only Latency Phase

In addition to the main experiment described above, a second, smaller data collection phase isolates the cryptographic cost of each hybrid group from network effects entirely. Where the main experiment varies device profile, network profile, and configuration together, this phase removes the network dimension: no tc-netem impairment is applied, so the client and server communicate over an unmodified loopback interface, and handshake latency reflects computational and OS/scheduling cost alone rather than network delay.

Each of the three hybrid groups (`x25519_mlkem512`, `X25519MLKEM768`, `SecP384r1MLKEM1024`) is measured directly, rather than through the four static/adaptive configurations used in the main experiment, across the same three device profiles. For each device–group pair, five warm-up trials are executed and discarded before measurement begins, ensuring that the cgroup and page cache are not cold for the first measured trial; the client process is then pinned to a single CPU core (`taskset -c 0`) for the duration of the phase to reduce scheduler-induced variance. As in the main experiment, the group–trial execution order is interleaved via a seeded shuffle (seed = 20260719) rather than run as sequential per-group blocks, for the same reason: preventing systematic drift from confounding one group with another.

Each trial records handshake latency, wire latency, handshake size, and success status. This produces

$$M_{\text{crypto}} = N_{device} \times N_{group} \times N = 3 \times 3 \times 50 = 450 \tag{3.7}$$

handshake measurements, written to `crypto_latency.csv`. This dataset is collected by the same run script as the main experiment, in a separate phase, and is analyzed in Chapter 4 to isolate the computational cost of each hybrid group and to verify device-tier enforcement independently of network conditions.

3.4 Performance Metrics

This section defines the quantitative performance metrics used to evaluate Adaptive-QKE across the experimental conditions defined in Section 3.3.4. The selected metrics capture the three principal dimensions of TLS handshake performance affected by hybrid post-quantum key exchange: handshake latency, CPU time, and handshake size. Each metric is defined formally, justified in terms of its relevance, and accompanied by a description of the measurement methodology.

3.4.1 TLS Handshake Latency

3.4.1.1 Definition

TLS handshake latency L is defined as the total elapsed time from the moment a client initiates a TLS connection to the moment the handshake is complete and application data can be exchanged, formally defined as:

$$L = T_{complete} - T_{init} \tag{3.8}$$

where T_{init} is the timestamp recorded immediately before the ClientHello message is sent, and $T_{complete}$ is the timestamp recorded immediately after the client processes the server's Finished message. Across N trials, the reported location statistic is the median:

$$\tilde{L} = \text{median}(L_1, \ldots, L_N) \tag{3.9}$$

Uncertainty on $\tilde{L}$ is quantified via bootstrap 95% confidence intervals, computed during analysis in Chapter 4.

3.4.1.2 Justification

Handshake latency is the most user-visible performance indicator in TLS deployments, as it directly determines the connection setup delay before any application data can flow. In hybrid post-quantum TLS, latency is influenced by both the computational cost of KEM operations, and the transmission time of the larger key material, both of

which scale with the parameter set and interact with network conditions. Studies have shown that at high packet loss rates, post-quantum handshake latency can be 6 to 8 times greater than classical equivalents due to retransmissions of large key material [13], making latency particularly sensitive to constrained network conditions. This variability is precisely what motivates context-aware group selection, making latency the primary metric for evaluating AdaptiveQKE.

3.4.1.3 Measurement

Latency is measured using application-level timestamps, capturing the interval between connection initiation and receipt of the server Finished message, derived from state-transition markers in the underlying OpenSSL client's diagnostic trace output. Timestamps are obtained using a high-resolution monotonic clock via Python's `time.perf_counter()`, applied as each trace line is read. This approach is preferred for latency measurement as it provides direct access to handshake boundaries within the client process, avoiding the overhead of trace parsing from an external packet capture. A relay-based wire-level timestamp, L_{wire}, measured independently from the first client-to-server byte to the last relayed byte at handshake completion (Section 3.4.3, byte-counting relay), serves as cross-validation for the application-level measurement. Because the relay runs outside any cgroup, L_{wire} is unaffected by CPU throttling on the client, allowing it to detect cases where CFS scheduling stalls would otherwise corrupt L. Accordingly, L is the primary latency metric wherever the client is unconstrained, whereas *Lwire* is treated as primary for any comparison involving the constrained or mid-range device tiers, in which CFS throttling acts directly on the measuring process. Chapter 4 states which metric is used in each analysis. Because this relay sits between the client and the server, handshake traffic crosses the shaped loopback interface (Section 3.3.4.2) multiple times per connection rather than once each way, and the configured delay and loss are applied at every crossing. Absolute latency values in Chapter 4 are larger than a single-crossing path would produce as a result; because every configuration is measured through the same relay, comparisons between configurations remain valid.

3.4.1.4 Latency Decomposition

The propagation delay imposed by a network profile is the same for every configuration measured under it, and it is large enough to dominate the differences of interest. Subtracting it leaves the residual latency

$$R = L_{\text{wire}} - k\,d \tag{3.10}$$

where d is the configured one-way delay (10, 35 or 60 ms) and k is the number of times a packet crosses the shaped loopback interface per round trip. Because the byte-counting relay sits in the path, a round trip crosses that interface four times rather than twice, so k is fixed at 4, reflecting the relay's known path topology rather than an empirically estimated value. Only propagation is removed: link rate and packet loss remain, so serialization and retransmission time are still present in R, which is therefore expected to differ between network profiles. Separating computation from transfer requires removing the network entirely, which is the purpose of the crypto-only phase (Section 3.3.4.6). Residual latency is a derived quantity used in the analysis, not a fourth performance metric.

3.4.2 CPU Time

3.4.2.1 Definition

CPU time U is defined as the total processor time consumed in executing a single TLS handshake trial, measured as the sum of user-mode and kernel-mode time across both the measuring process and any subprocess performing the handshake, since the cryptographic operations execute inside a spawned OpenSSL subprocess rather than the measuring process itself:

$$U = \Delta u_{\text{self}} + \Delta s_{\text{self}} + \Delta u_{\text{child}} + \Delta s_{\text{child}} \tag{3.11}$$

where Δu and Δs denote the user-mode and kernel-mode time deltas, respectively, taken immediately before and after the handshake for the measuring process (*self*)

and its child process (*child*). Across N trials, the reported location statistic is the median:

$$\tilde{U} = \text{median}(U_1, \ldots, U_N) \tag{3.12}$$

Uncertainty on $\tilde{U}$ is quantified via bootstrap 95% confidence intervals, computed during analysis in Chapter 4.

3.4.2.2 Justification

CPU time isolates the computational component of handshake performance independently of network conditions, making it particularly relevant in heterogeneous deployments where post-quantum computational demands vary significantly across device classes. For example, on resource-constrained devices, excessive CPU load during the handshake may degrade system responsiveness by starving other processes, and may increase the risk of denial-of-service under high connection rates. This computational cost, rather than network transmission, is the dominant contributor to the overhead reported for constrained post-quantum TLS deployments [14], motivating CPU time as a metric that isolates it directly.

3.4.2.3 Measurement

CPU time is measured using `getrusage()` deltas taken immediately before and after each handshake trial, scoped to the measuring process and summed with its child process to capture the OpenSSL subprocess in which the cryptographic operations actually execute. This approach isolates the client-side computational cost from system-wide CPU activity without the sampling-interval trade-offs of interval-based profiling.

3.4.3 TLS Handshake Size

3.4.3.1 Definition

TLS handshake size S is defined as the total number of bytes exchanged between the client and the server during the handshake phase, from the first client-to-server byte

to the last byte relayed at handshake completion:

$$S = B_{c2s} + B_{s2c} \tag{3.13}$$

where B_{c2s} and B_{s2c} are the total bytes observed in the client-to-server and server-to-client directions, respectively, at the byte-counting relay (Section 3.4.3, Measurement). Across N trials, the reported location statistic is the median:

$$\tilde{S} = \text{median}(S_1, \ldots, S_N) \tag{3.14}$$

Uncertainty on $\tilde{S}$ is quantified via bootstrap 95% confidence intervals, computed during analysis in Chapter 4.

3.4.3.2 Justification

Handshake size is a direct and deterministic consequence of the key and ciphertext sizes introduced by the chosen hybrid group, making it a precise and stable indicator of communication overhead across configurations. Unlike latency and CPU time, which are subject to variance from network conditions and system load, handshake size is a deterministic function of the chosen hybrid group's key and ciphertext sizes, making it a stable reference metric across experimental conditions. On bandwidth-constrained or high-latency links, the primary concern is whether the combined handshake size exceeds the TCP initial congestion window, forcing additional round trips and disproportionately increasing latency; this behaviour is confirmed in TLS 1.3 deployments with large post-quantum key material [13].

3.4.3.3 Measurement

Handshake size is measured by a byte-counting relay positioned between the client and the server, which counts every byte forwarded in each direction and snapshots the running totals once the wire falls quiescent after handshake completion. Because this approach counts raw bytes rather than parsing individual TLS record contents,

it requires no decryption of the captured traffic and avoids the packet-capture race conditions associated with tools such as tshark on a loopback interface.

3.4.4 AdaptiveQKE Performance Testing Metrics

Table 3.8 summarizes the three performance metrics, their units, the aspect of handshake performance each captures, and the measurement method used to collect them.

Table 3.8: Summary of performance metrics used to evaluate AdaptiveQKE

Metric	**Unit**	**Aspect Captured**	**Measurement Method**
TLS Handshake Latency	ms	Connection setup delay from computational and network costs	`time.perf_counter()` (OpenSSL trace); byte-counting relay (cross-validation)
CPU Time	ms	Processor time consumed on client during handshake (self + child process)	`getrusage()` deltas
TLS Handshake Size	bytes	Total data exchanged; reflects key material overhead	Byte-counting relay (no decryption required)

Evaluating all three across the full experimental space defined in Section 3.3.4 enables the comparative analysis in Chapter 4 to characterise not only whether AdaptiveQKE reduces overall overhead, but also which dimension of overhead it reduces most effectively and under which conditions the reduction is most significant.

3.5 Summary

This chapter presented the design and methodology of AdaptiveQKE, a policy-driven pre-handshake selection mechanism for hybrid post-quantum key exchange in TLS 1.3. The proposed approach addresses the gap between the performance variability of hybrid key exchange across heterogeneous environments and the lack of a context-aware selection mechanism in the TLS 1.3 standard.

At runtime, the mechanism operates in two stages. In the pre-handshake selection stage, a lightweight context probe collects runtime information, including RTT and device capability indicators, which are processed by a policy engine to select an appropriate hybrid group from a predefined allow-list. In the handshake execution stage, the TLS 1.3 handshake is performed using the selected configuration without modifying the standard protocol flow. These two runtime stages are distinct from the two methodological phases defined in Section 3.2.

The system architecture consists of three client-side components — the context probe, the policy engine, and the TLS client — together with a standard, unmodified TLS 1.3 server. The implementation relies on OQS-enabled OpenSSL with the OQS provider, which provides support for ML-KEM and hybrid key exchange groups. The policy engine applies a two-stage decision process, combining RTT-driven selection with device-aware validation to ensure both performance efficiency and computational feasibility. A conceptual third stage, applying application-declared bounds on the selected security level, is included in the design but not applied in the experimental evaluation.

The evaluation methodology defines three device profiles and three network condition profiles, resulting in nine experimental scenarios. Four configurations — three static baselines and the proposed adaptive approach — are evaluated using three performance metrics: handshake latency, CPU time, and handshake size. Measurements are collected over 50 trials per condition, yielding a total of 1,800 handshake instances.

The design choices adopted in this chapter, including the pre-handshake selection layer, the threshold-based policy, and the selection of hybrid groups spanning multiple NIST security levels, collectively enable a systematic and fair comparison between static and adaptive approaches. The results of this evaluation are presented and analysed in Chapter 4.

Chapter 4

Results & Analysis

4.1 Introduction

This chapter presents and analyzes the experimental results obtained from the evaluation of the proposed AdaptiveQKE framework. The evaluation investigates the behavior of adaptive hybrid group selection under heterogeneous endpoint capabilities and network characteristics and compares its performance with the three static hybrid configurations considered in this study.

The analysis focuses on the main performance metrics defined in the experimental methodology, including TLS handshake latency, CPU time, and TLS handshake size. In addition, the chapter examines the reliability of the collected measurements, the behavior and overhead of the adaptive selection policy, and the impact of device and network constraints on the evaluated hybrid groups.

The results are analyzed across the different device and network profiles to determine how the selected hybrid configuration affects overall TLS 1.3 handshake performance and to identify the conditions under which adaptive selection provides measurable advantages or introduces additional overhead. The chapter concludes by interpreting these findings in relation to the objectives of AdaptiveQKE and the trade-offs involved in context-aware hybrid group selection.

4.2 Experimental Execution

This section describes how the proposed AdaptiveQKE framework was executed to obtain the experimental data analyzed in this chapter. The experiments were conducted using the implementation presented in Chapter 3 under controlled and repeatable conditions, ensuring that all evaluated configurations were executed within the same software environment and experimental workflow.

The implementation was based on OpenSSL with the OQS Provider to support TLS 1.3 hybrid key exchange. A dedicated TLS server was configured to accept only the three hybrid groups evaluated in this study, namely `x25519_mlkem512`, `X25519MLKEM768`, and `SecP384r1MLKEM1024`. For each trial, the client established a complete TLS 1.3 handshake with the server while the framework automatically collected the required measurements, including handshake latency, wire latency, CPU time, and handshake size (section 3.4).

To evaluate the framework under different execution conditions, both device capabilities and network characteristics were enforced programmatically rather than assumed. Device profiles were created using Linux cgroup v2 CPU and memory limits to represent High-performance, Mid-range, and Constrained execution environments. Network profiles were implemented using `tc-netem`, where propagation delay, available bandwidth, and packet loss were configured to reproduce Low-latency, Moderate, and High-latency network conditions. This approach ensured consistent execution conditions throughout the entire evaluation process.

During every experimental trial, the framework first collected the runtime context by measuring the round-trip time (RTT) and obtaining the available CPU share and memory limits. These context parameters were then supplied to the adaptive policy engine, which selected the appropriate hybrid group according to the two-stage decision mechanism described in Chapter 3. The selected group was subsequently used to execute the TLS 1.3 handshake, after which all measured values were automatically recorded in the experiment dataset for later analysis.

The complete evaluation consisted of three device profiles, three network profiles, and four configuration modes (Adaptive, Static-512, Static-768, and Static-1024). Each

configuration was repeated for fifty independent trials under every device-network combination, resulting in a total of 1,800 TLS handshake measurements that formed the basis of the analysis presented in the following sections. Within each device-network condition, trials from all four configurations were interleaved in a randomized, seeded order rather than executed as contiguous per-configuration blocks, so that any systematic drift over the course of a run (e.g. thermal effects, cgroup warm-up) could not bias one configuration more than another; the execution position of each trial was recorded to allow this to be verified directly in Section 4.3.

4.3 Data Validity Checks

Before analyzing the experimental results, the collected dataset was verified to ensure its completeness, consistency, and suitability for statistical analysis. Table 4.1 summarizes the validation checks performed on the complete dataset.

Table 4.1: Summary of dataset validation checks

Gate	**Value**
Main trials successful	1800/1800
Crypto-phase trials successful	450/450
Trials excluded from analysis	0
Classified == enforced profile	1800/1800
Drift corr(exec_order, wire): mean over cells	0.0008
Drift corr(exec_order, wire): median over cells	-0.0354
Outliers overall (Tukey 1.5*IQR per cell)	7.70%
Max median shift if outliers were removed	8.18 ms
Mean median shift if outliers were removed	0.47 ms

Table 4.1 confirms that all 1,800 TLS handshake trials and 450 crypto-only trials completed successfully, resulting in a 100% execution success rate. No experimental runs were discarded because of execution failures or missing measurements. Furthermore, all device and network profiles were correctly identified throughout the evaluation, confirming that the experimental conditions were applied as intended. The stability checks also indicated that no significant measurement drift occurred

during data collection, consistent with the interleaved trial ordering described in Section 4.2. The median sensitivity analysis showed that excluding outliers did not materially affect the reported results overall, with one exception: the Constrained device under the High-latency network for the Adaptive configuration, where removing seven outlier trials shifted the median by the full 8.18 ms reported above. This cell is addressed directly in Section 4.5. These observations indicate that the collected dataset is internally consistent and suitable for the performance analysis presented in the following sections.

To further evaluate data quality, the relationship between packet loss and statistical outliers was examined for each network profile, as summarized in Table 4.2.

Table 4.2: Relationship between configured packet loss and statistical outliers

Network	**Configured Loss (%)**	**Outlier Trials**	**Of**	**Outlier (%)**
Low-latency	0	15	600	2.5
Moderate	0.5	43	600	7.2
High-latency	2	80	600	13.3

The results show that the percentage of outliers increased gradually as network conditions became more challenging. The Low-latency profile produced the smallest proportion of outliers, while the High-latency profile exhibited the largest percentage. This behaviour is consistent with the expected increase in timing variability under more constrained network conditions. Nevertheless, the overall proportion of outliers remained limited and did not compromise the stability or validity of the collected dataset.

Overall, the validation procedures confirm that the experimental dataset is complete, internally consistent, and sufficiently stable to support the comparative performance analysis presented in the remainder of this chapter.

4.4 Selection Accuracy and Policy Overhead

This section evaluates the correctness of the adaptive policy decisions and the runtime overhead introduced by the context-aware selection process. Before comparing the communication performance of AdaptiveQKE with the static configurations, it is necessary to verify two aspects of the framework. First, the adaptive policy must consistently select the hybrid group defined by the decision rules under each device–network condition. Second, the additional cost introduced by context acquisition and policy evaluation must be quantified separately in order to distinguish the computational cost of the decision mechanism from the communication cost of runtime probing.

4.4.1 Selection Accuracy

Table 4.3 presents the hybrid groups selected by the adaptive policy under each device–network combination. For every evaluated condition, the observed selection matched the expected decision defined by the policy rules presented in Chapter 3. Each of the nine device–network scenarios therefore achieved 50 correct selections out of 50 trials, corresponding to an overall selection accuracy of 100%.

Table 4.3: Adaptive policy selection accuracy across device–network conditions

Device	**Network**	**Expected Group**	**Selected as Expected**
High-performance	Low-latency	1024	50/50
High-performance	Moderate	768	50/50
High-performance	High-latency	512	50/50
Mid-range	Low-latency	768	50/50
Mid-range	Moderate	768	50/50
Mid-range	High-latency	512	50/50
Constrained	Low-latency	512	50/50
Constrained	Moderate	512	50/50
Constrained	High-latency	512	50/50

These results confirm that the adaptive policy engine consistently interpreted the collected runtime context and applied the intended decision rules without incorrect or inconsistent selections. Therefore, the performance differences examined in the following sections can be attributed to the behaviour of the selected configurations rather than to implementation errors in the policy logic.

4.4.2 Policy Overhead

The adaptive selection process consists of two distinct sources of overhead: the time required to acquire the runtime context and the computational cost required to evaluate the policy itself. Table 4.4 reports these two components separately across the three evaluated network profiles.

Table 4.4: Context probing and policy evaluation overhead across network conditions (all time values in ms)

Network	**RTT Median**	**Probe Median**	**Probe 95% CI**	**Probe Max**	**Trials >500 ms**	**Policy Median**
Low-latency	20.9	21.6	[21.6, 21.7]	23.5	0	0.026
Moderate	71.0	71.8	[71.7, 71.9]	75.4	0	0.026
High-latency	121.4	122.2	[122.1, 122.2]	126.5	0	0.027

The measurements show a clear distinction between context-acquisition cost and policy-computation cost. The median context-probing cost increased from 21.6 ms under the Low-latency profile to 71.8 ms under the Moderate profile and 122.2 ms under the High-latency profile. This increase is expected because the probing procedure includes RTT measurement and therefore depends directly on the underlying network delay. In contrast, the median policy-evaluation cost remained nearly constant at approximately 0.026–0.027 ms across all three network conditions. No probing trial exceeded 500 ms. These results demonstrate that the decision logic itself introduces negligible computational overhead, whereas the principal runtime cost of the adaptive process is associated with acquiring the network context.

Accordingly, the overhead of AdaptiveQKE should not be interpreted as a consequence of the policy computation. The dominant additional cost occurs before the policy decision, during context probing, and its magnitude is primarily determined by the current network condition.

4.4.3 Total First-Connection Cost

To quantify the practical effect of the context-probing phase, Table 4.5 combines the AdaptiveQKE wire latency with the corresponding probing cost to obtain the total latency incurred during the first connection. The resulting value is compared with both the matching static configuration and the Static-1024 baseline.

Table 4.5: Total first-connection cost across device–network conditions, with all latency values reported in milliseconds

Device	**Network**	**Adaptive Wire**	**Probe**	**Total**	**Matching Static**	**Static-1024**
High-performance	Low-latency	53.7	21.6	75.2	53.5	53.5
High-performance	Moderate	148.6	71.8	220.5	148.5	158.2
High-performance	High-latency	260.2	122.2	382.5	260.1	276.3
Mid-range	Low-latency	44.9	21.6	66.5	45.0	58.5
Mid-range	Moderate	148.6	71.8	220.5	148.6	158.8
Mid-range	High-latency	260.2	122.2	382.4	260.3	278.2
Constrained	Low-latency	56.9	21.7	78.7	48.8	89.1
Constrained	Moderate	149.7	71.7	221.8	148.7	189.6
Constrained	High-latency	277.1	122.1	399.1	268.7	308.6

The results demonstrate that context acquisition introduces a measurable one-time penalty during the first connection. In every evaluated device–network condition, the AdaptiveQKE total first-connection latency is higher than the wire latency of the

corresponding matching static configuration because the adaptive value includes the additional probing phase.

The magnitude of this difference increases with network latency. For the High-performance device, for example, the total AdaptiveQKE first-connection latency increases from 75.2 ms under the Low-latency profile to 220.5 ms under the Moderate profile and 382.5 ms under the High-latency profile. The corresponding matching-static latencies are 53.5 ms, 148.5 ms, and 260.1 ms, respectively. A similar pattern is observed for the Mid-range and Constrained device profiles.

The comparison with Static-1024 leads to the same general conclusion. Although selecting a lighter hybrid group may reduce the wire-level handshake cost in some device–network conditions, the additional probing time can outweigh this reduction during the initial connection. For example, under the Constrained High-latency condition, AdaptiveQKE records a total first-connection latency of 399.1 ms, compared with 308.6 ms for Static-1024.

These results show that AdaptiveQKE should not be interpreted as optimizing the latency of the first connection. Instead, the first connection exposes the cost of acquiring the runtime context required for adaptation. Once the context has been collected and a hybrid group has been selected, the probing cost can be separated from subsequent handshake measurements, allowing the effect of the selected configuration itself to be evaluated independently.

Overall, the adaptive policy achieved 100% selection accuracy and introduced negligible policy-computation cost. However, the context-probing phase introduces a non-negligible one-time first-connection overhead whose magnitude increases with network latency. This distinction between policy-computation overhead and context-acquisition overhead is essential when interpreting the comparative results presented in the following sections.

4.5 Post-Selection Consistency with the Matching Static Configuration

This section examines whether AdaptiveQKE introduces a measurable change in TLS handshake latency after the policy has selected a key-exchange group. For each device–network condition, the Adaptive configuration was compared with the static configuration that enforced the same group selected by the policy. Table 4.6 reports the median wire latency for both configurations, together with the difference in medians and its 95% bootstrap confidence interval. The difference is calculated as AdaptiveQKE minus static; therefore, a positive value indicates higher latency under AdaptiveQKE.

For the high-performance device, the policy selected groups 1024, 768, and 512 under the low-, moderate-, and high-latency networks, respectively. The corresponding differences between the AdaptiveQKE and matching static medians were +0.16, +0.09, and +0.10 ms. For the mid-range device, the selected groups were 768, 768, and 512, with median differences of −0.09, −0.03, and −0.18 ms, respectively. Across these six conditions, the absolute difference did not exceed 0.18 ms, and every 95% confidence interval spanned zero. These small differences were within the variability observed across repeated trials and therefore do not provide evidence of a consistent latency difference between the two configurations.

For the constrained device, the policy selected group 512 under all three network profiles. The AdaptiveQKE median exceeded the matching Static-512 median by 8.14 ms on the low-latency network, 0.91 ms on the moderate network, and 8.44 ms on the high-latency network. These point estimates were larger than those observed for the other devices, but every confidence interval still spanned zero, so none provide clear evidence of a systematic overhead. The two larger gaps have different origins. The high-latency case is driven by the seven outlier trials identified in Section 4.3; excluding them narrows the gap to under one millisecond, consistent with a data-quality artifact. The low-latency case contains no outliers, yet its interval remains wide; this instead reflects CPU-quota quantization on the constrained device, examined in Section 4.9.

Table 4.6: AdaptiveQKE wire latency compared with the matching static configuration, separated by device profile. Differences are calculated as AdaptiveQKE minus static; the intervals are 95% bootstrap confidence intervals for the median difference.

(a) High-performance device

Network profile	Static group	Adaptive median (ms)	Static median (ms)	Diff. (ms)	Diff. (%)	95% CI for difference (ms)
Low-latency	1024	53.7	53.5	+0.16	+0.3%	[−0.12, +0.44]
Moderate	768	148.6	148.5	+0.09	+0.1%	[−0.15, +0.36]
High-latency	512	260.2	260.1	+0.10	0.0%	[−0.13, +0.29]

(b) Mid-range device

Network profile	Static group	Adaptive median (ms)	Static median (ms)	Diff. (ms)	Diff. (%)	95% CI for difference (ms)
Low-latency	768	44.9	45.0	−0.09	−0.2%	[−0.34, +0.20]
Moderate	768	148.6	148.6	−0.03	0.0%	[−0.31, +0.19]
High-latency	512	260.2	260.3	−0.18	−0.1%	[−0.41, +0.20]

(c) Constrained device

Network profile	Static group	Adaptive median (ms)	Static median (ms)	Diff. (ms)	Diff. (%)	95% CI for difference (ms)
Low-latency	512	56.9	48.8	+8.14	+16.7%	[−0.08, +9.56]
Moderate	512	149.7	148.7	+0.91	+0.6%	[−0.32, +9.11]
High-latency	512	277.1	268.7	+8.44	+3.1%	[−6.18, +9.74]

In both cases, the point estimate alone should not be read as evidence of added latency from the Adaptive mechanism.

Overall, the 95% confidence intervals for the median differences spanned zero in all nine device–network conditions. Therefore, the results provide no clear evidence of a systematic post-selection wire-latency difference between AdaptiveQKE and a static configuration using the same key-exchange group. This outcome is consistent with the intended design, in which AdaptiveQKE changes only the group-selection decision. Once a group is selected, its post-selection wire latency is not measurably different from that of the matching static configuration. Consequently, performance differences in subsequent comparisons are interpreted primarily as effects of the selected key-exchange group rather than as additional latency introduced by the Adaptive mechanism itself.

Table 4.6 evaluates only the wire latency of the TLS handshake after group selection. It does not include the time required to measure the RTT or execute the selection policy; these pre-handshake costs were evaluated separately in Section 4.4. Finally, the absence of a detectable latency difference should not be interpreted as formal evidence that the AdaptiveQKE and static configurations are statistically equivalent.

4.6 Group Latency across Network Conditions

This section examines how network conditions affect the latency differences among the key-exchange groups. The overall wire-latency level was dominated by the configured network delay: the matching static medians were approximately 45–54 ms on the low-latency profile, 149 ms on the moderate profile, and 260–269 ms on the high-latency profile. This variation was substantially larger than the differences among groups, which appeared as smaller increments above the network-dependent baseline.

Table 4.7 and Figure 4.1 isolate the Static-768 minus Static-512 gap on the high-performance device. The observed gap was only 0.04 ms at 100 Mbit/s, with a confidence interval spanning zero. It increased to 0.58 ms at 15 Mbit/s and 3.08 ms at 3 Mbit/s, with both confidence intervals entirely above zero.

Table 4.7: Observed Static-768 minus Static-512 wire-latency gap on the high-performance device, by link rate.

Network	**Rate (Mbit/s)**	**Observed gap (ms)**	**95% CI (ms)**
Low-latency	100	0.04	[−0.13, +0.23]
Moderate	15	0.58	[+0.31, +0.80]
High-latency	3	3.08	[+2.78, +3.32]

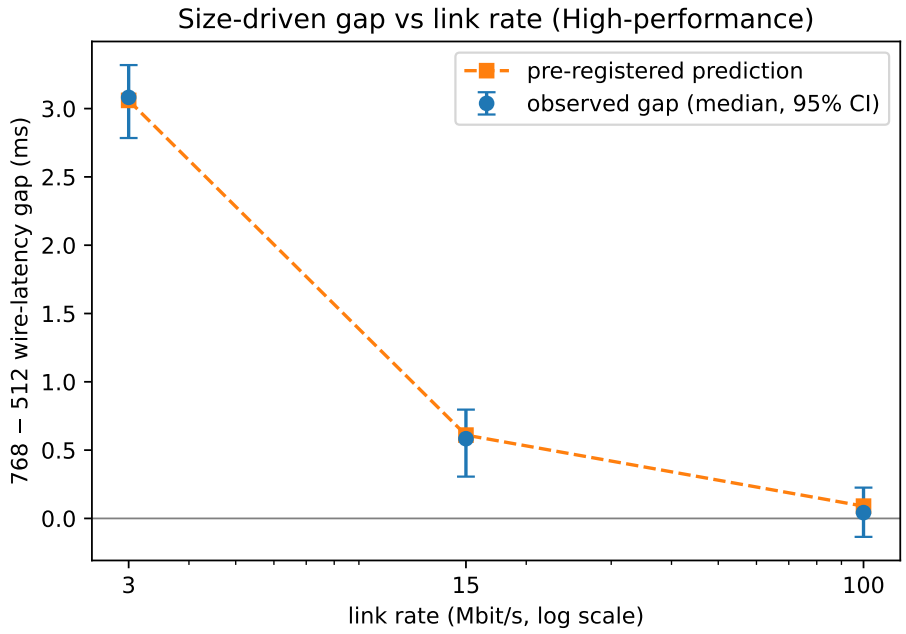


Figure 4.1: Observed Static-768 minus Static-512 wire-latency gap on the high-performance device, by link rate; error bars show 95% confidence intervals.

The widening gap is consistent with the handshake sizes of 3176, 3892, and 4886 bytes for groups 512, 768, and 1024, respectively. Transmitting an additional ΔB bytes over a link with rate R bits/s has the first-order cost

$$\Delta t_{\text{tx}} = \frac{8\Delta B}{R}.$$

Additional bytes therefore have negligible effect on a high-rate link but become increasingly costly as the rate falls. This relation explains the observed trend: the gap widens as link rate falls, consistent with a transmission-cost mechanism rather

than a fixed per-connection processing cost. The full group results showed the same general behavior, with Static-1024 remaining above the two smaller groups and its margin widening under rate limitation.

The no-network baseline in Table 4.9 clarifies that the transmission-based interpretation applies directly to the Static-768 minus Static-512 gap. Both groups produced a median of 3.17 ms at the reported precision, with a median difference of 0.00 ms and a 95% confidence interval of [−0.06, +0.15] ms. Their widening separation across the network profiles therefore cannot be explained by a fixed local-processing difference and is consistent with the transmission cost of the additional handshake bytes under rate limitation.

Static-1024 behaved differently. Its no-network median was 12.27 ms, which was 9.09 ms above Static-512, with a 95% confidence interval of [+8.97, +9.23] ms. Its higher wire latency therefore reflects both a substantial no-network processing cost and the transmission cost of its larger handshake. These computational and device-related effects are examined further in Section 4.9.

Souvatzidaki and Limniotis [13] report that post-quantum handshake latency under high packet loss can reach 6–8× that of classical handshakes, driven by retransmission of large key material. The present results do not reproduce an effect of this magnitude: even the largest evaluated group (`SecP384r1MLKEM1024`, 4,886 bytes) remains well within the TCP initial congestion window (approximately 14,600 bytes for 10 MSS), so no configuration in this experiment required additional round trips due to handshake size alone. The severe degradation Souvatzidaki and Limniotis report also emerges most strongly at loss rates above those tested here (up to 12%, versus 2% in the High-latency profile). The two results are therefore complementary rather than conflicting: this experiment isolates the byte-transmission cost of hybrid group selection within a single-round-trip regime, while their finding describes what happens once handshake size or loss severity pushes a connection outside that regime.

Overall, network conditions dominated the total wire-latency level. The additional latency of Static-768 relative to Static-512 was primarily rate-dependent and consistent with transmission cost, whereas Static-1024 also exhibited a measurable no-network processing cost. These results support network-aware group selection while preserving

a separate device stage to account for computational limitations.

4.7 Separating Network Delay from Key-Exchange Cost

To separate propagation delay from the remaining handshake cost, residual latency was calculated as

$$L_{\text{res}} = L_{\text{wire}} - 4d,$$

where L_{wire} is the measured wire latency and d is the configured one-way delay. The term $4d$ represents the propagation component assigned by the measurement model to the handshake path. The resulting residual is not a pure cryptographic measurement; it also includes transmission, protocol processing, and operating-system overhead.

Table 4.8: Residual wire latency after subtracting the modeled propagation component on the high-performance device.

Group	**Network profile**	**Median (ms)**	**95% CI (ms)**
512	Low-latency	4.9	[4.7, 5.0]
512	Moderate	8.0	[7.9, 8.2]
512	High-latency	20.1	[20.0, 20.3]
768	Low-latency	4.9	[4.8, 5.1]
768	Moderate	8.5	[8.4, 8.7]
768	High-latency	23.2	[23.0, 23.4]
1024	Low-latency	13.5	[13.3, 13.6]
1024	Moderate	18.2	[18.0, 18.4]
1024	High-latency	36.3	[36.0, 36.7]

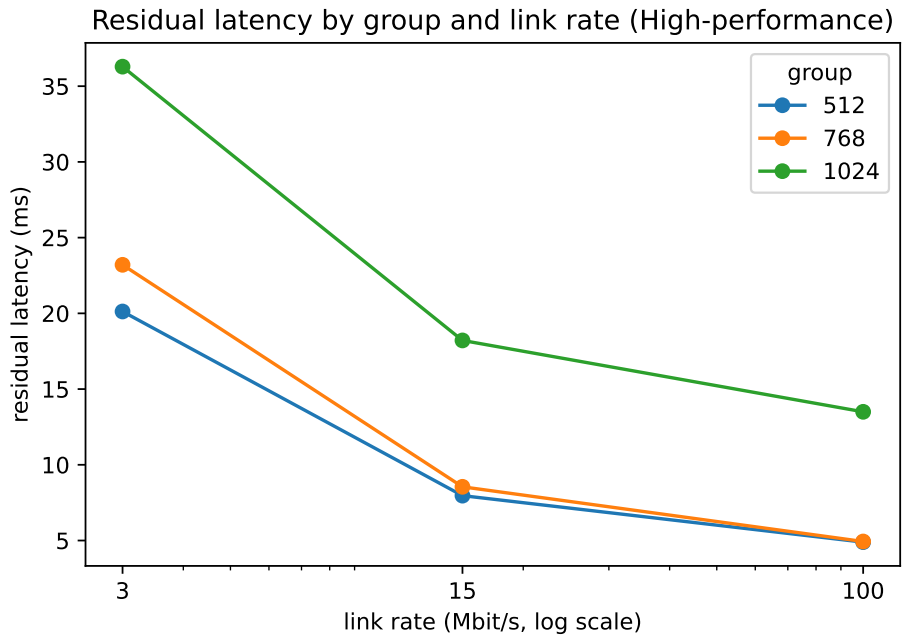


Figure 4.2: Residual wire latency by key-exchange group and link rate on the high-performance device. The link-rate axis uses a logarithmic scale.

Table 4.8 and Figure 4.2 show that the residual latency increased as the link rate decreased for all three groups. At 100 Mbit/s, groups 512 and 768 both had a residual median of 4.9 ms, whereas group 1024 had a median of 13.5 ms. At 3 Mbit/s, the corresponding medians increased to 20.1, 23.2, and 36.3 ms. Thus, reducing the link rate from 100 to 3 Mbit/s increased the residual by 15.2 ms for group 512, 18.3 ms for group 768, and 22.8 ms for group 1024.

The higher and steeper curve for group 1024 reflects two distinct effects. First, it retained an 8.6 ms margin over group 512 even at 100 Mbit/s. This is consistent with the no-network baseline in Table 4.9, where group 1024 was 9.09 ms above group 512. Its higher residual therefore includes a substantial local processing component that remains after propagation delay is removed. Second, its margin over group 512 increased to 16.2 ms at 3 Mbit/s. Because group 1024 has the largest handshake size, its transmission cost increases more rapidly as the available link rate decreases. The curve is therefore higher because of its local processing baseline and steeper because of the rate-dependent cost of transmitting its additional bytes.

Removing propagation delay confirms that the large increase in total wire latency across the network profiles should not be attributed entirely to key-exchange processing. It also shows that the remaining group-dependent cost is not uniform: the 512–768 difference is primarily rate-dependent, whereas group 1024 combines a measurable local processing cost with a larger transmission cost. This distinction supports considering both network conditions and device capability when selecting the key-exchange group.

4.8 Handshake Cost without Network Delay

This section evaluates the TLS handshake cost of the three static hybrid groups without any emulated network delay. Measurements were collected during the crypto-latency phase over the loopback interface and the High-performance device profile was used, meaning that no artificial CPU or memory constraints were applied.

Table 4.9: Baseline TLS handshake latency under the High-performance profile without emulated network delay ($n = 50$ per group).

Hybrid group	**Handshake size (bytes)**	**Median latency (Q1–Q3) ms**	**Difference vs. x25519_mlkem512 (95% CI) ms**
`x25519_mlkem512`	3,176	3.175 (3.121–3.307)	—
`X25519MLKEM768`	3,892	3.171 (3.107–3.418)	−0.004 ms (−0.058 to +0.146)
`SecP384r1MLKEM1024`	4,886	12.268 (11.994–12.666)	+9.093 (+8.967 to +9.227)

The table reports the median latency (Q1–Q3) for each group. Differences relative to the 512 group are presented with 95% confidence intervals. The results in Table 4.9 indicate that the 512 and 768 groups produced very similar median handshake latencies of 3.175 ms and 3.171 ms. Although the handshake size of the 768 group was 716 bytes larger, its estimated median difference relative to 512 was only −0.004 ms. The corresponding 95% confidence interval ranged from −0.058 to +0.146 ms and included zero. Therefore, no clear median latency difference was detected between 512 and 768 under this baseline condition. The small negative point estimate does not indicate

that 768 is faster, nor does this result imply that the computational costs of the two groups are identical.

This is consistent with the two groups undergoing the same complete certificate-based TLS handshake, comprising the ClientHello, ServerHello, Certificate, CertificateVerify, and Finished messages, in addition to shared OpenSSL processing. Any underlying computational difference between ML-KEM-512 and ML-KEM-768 is therefore likely small relative to this shared overhead, and may fall below what this experiment's measurement resolution and run-to-run variability, including OS scheduling and VM-related noise from the VMware environment, can reliably detect. A `speed_kem` reference measurement, captured on the same environment immediately before the main experiment, supports this interpretation: ML-KEM-768 required approximately 52-65% more time than ML-KEM-512 across KeyGen, Encapsulation, and Decapsulation, corresponding to an absolute difference of roughly 5-8 μs per operation. This is consistent with a real but small underlying computational difference, several orders of magnitude below the millisecond-scale handshake latency reported in Table 4.9, and therefore not statistically distinguishable at the handshake level with the present sample size. This is further consistent with Zheng et al. [18], who found that optimizing ML-KEM implementation alone yields minimal improvement to overall hybrid handshake performance, since latency in hybrid mode depends on both the classical and post-quantum components rather than either in isolation. The implications of this virtualized measurement environment for the external validity of these results are discussed in Section 5.6.

In contrast, the 1024 group produced a median handshake latency of 12.268 ms, which was 9.093 ms higher than the 512 baseline. Its 95% confidence interval ranged from +8.967 to +9.227 ms and remained entirely above zero, demonstrating a clear latency increase. However, this result represents the complete `SecP384r1MLKEM1024` hybrid group. Since this group combines ML-KEM-1024 with SecP384r1, rather than X25519, the measured increase cannot be attributed solely to ML-KEM-1024.

4.9 Effect of Device Constraints

This section examines how device resource constraints affected TLS handshake latency. The three static configurations were evaluated across the High-performance, Mid-range, and Constrained device profiles under the same Low-latency network condition.

Table 4.10: Median Wire Latency Across Device Profiles Under the Low-Latency Network Profile.

Hybrid group	**High-performance (ms)**	**Mid-range (ms)**	**Constrained (ms)**	**Con/Mid ratio**
`x25519_mlkem512`	44.9	44.9	48.8	1.09
`X25519MLKEM768`	44.9	45.0	57.4	1.28
`SecP384r1MLKEM1024`	53.5	58.5	89.1	1.52

As shown in Table 4.10, `x25519_mlkem512` and `X25519MLKEM768` produced almost the same median wire latency under the High-performance and Mid-range profiles, at approximately 45 ms. Under the Constrained profile, the latency increased to 48.8 ms for `x25519_mlkem512` and 57.4 ms for `X25519MLKEM768`. `SecP384r1MLKEM1024` was more affected by the constraints, increasing from 58.5 ms in the Mid-range profile to 89.1 ms in the Constrained profile. Overall, stronger device constraints had a clearer effect on the heavier hybrid configurations.

Table 4.11: Crypto-Phase Handshake Latency Across Device Profiles, with Enforcement Evidence

Device profile	**Quota (%)**	**512 (ms)**	**768 (ms)**	**1024 (ms)**	**Slowdown (512)**	**Trials within 1 ms of a 10 ms boundary (%)**
High-performance	none	3.2	3.2	12.3	1.00×	0%
Mid-range	25	9.4	9.1	19.4	2.98×	79%
Constrained	10	20.5	29.8	50.2	6.47×	97%

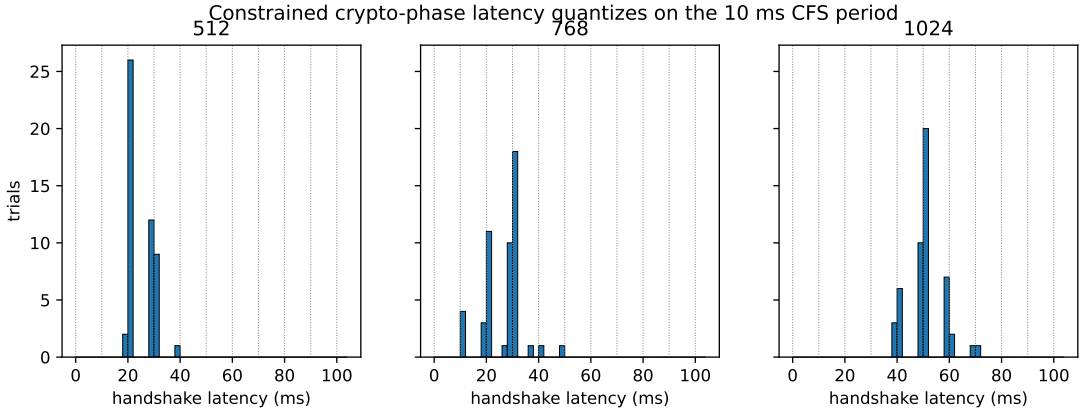


Figure 4.3: Crypto-phase handshake latency on the Constrained device, by hybrid group. Dotted lines mark multiples of the 10 ms CFS scheduling period.

Together, Tables 4.10 and 4.11 provide two complementary views of device-resource limitations. Table 4.10 shows their effect on the packet-observed TCP/TLS connection latency under a low-latency network condition, whereas Table 4.11 focuses on the client-observed handshake latency without network emulation. Both tables show that the latency increases as device resources become more restricted. The latency increase was most pronounced for `SecP384r1MLKEM1024`, followed by `X25519MLKEM768`, while `x25519_mlkem512` showed the smallest increase. This indicates that resource limitations have a stronger effect on configurations that require more computational work.

The slowdown and boundary-hit columns in Table 4.11 make the mechanism behind this pattern explicit rather than descriptive. The share of trials landing within 1 ms of a 10 ms CFS-period boundary rises from 0% on the unconstrained device to 79% on Mid-range and 97% on Constrained, confirming that the enforced CPU quota, not incidental system noise, is responsible for the latency increase. On the Constrained device, the median `x25519_mlkem512` handshake completes almost exactly at 2 full CFS scheduling windows, indicating that even the lightest group is regularly paused and resumed by the scheduler at least once under this quota.

Figure 4.3 shows this directly: on the Constrained device, latency for each group clusters in discrete steps near multiples of the 10 ms period rather than varying continuously, consistent with the handshake process being repeatedly paused and

resumed once its CPU quota is exhausted within a scheduling period. The clustering is most pronounced for `SecP384r1MLKEM1024`, whose heavier computation exhausts the quota more often within a single handshake, pushing more trials into higher CFS windows; `x25519_mlkem512` remains concentrated closer to a single low window because its lighter computation less often exceeds an additional quota period.
Table 4.10 shows the final practical effect in a networked TCP/TLS connection, with median latency values ranging from approximately 45 ms to 89 ms. Table 4.11, together with Figure 4.3, further indicates that device-resource limitations contribute to the observed latency increase through a specific, verifiable mechanism, CPU-quota quantization, rather than the increase being caused by network conditions alone.

4.10 Adaptive vs. the Static Baselines

This section evaluates the Adaptive policy against the static hybrid configurations. The comparison examines whether Adaptive can reduce CPU time and handshake size by selecting an appropriate hybrid configuration based on observed network conditions and the available device resources.

Table 4.12: Adaptive Savings Relative to SecP384r1MLKEM1024 Across Device and Network Profiles

Device profile	**Network profile**	**Wire saving (%)**	**CPU-time saving (%)**	**Size saving (%)**
High-performance	Low-latency	−0.3	−0.3	0
High-performance	Moderate	6.1	17.7	20.3
High-performance	High-latency	5.8	18.4	35.0
Mid-range	Low-latency	23.2	18.3	20.3
Mid-range	Moderate	6.5	18.4	20.3
Mid-range	High-latency	6.5	18.9	35.0
Constrained	Low-latency	36.1	20.7	35.0
Constrained	Moderate	21.0	18.7	35.0
Constrained	High-latency	10.2	17.4	35.0
Mean	—	**12.8**	**16.5**	**26.2**

Table 4.12 shows the savings achieved by the Adaptive policy relative to the static `SecP384r1MLKEM1024` configuration across different device-resource and network profiles. On average, Adaptive reduced wire time by 12.8%, CPU time by 16.5%, and handshake size by 26.2%. The largest wire-time saving was observed on the Constrained device under the Low-latency network profile, reaching 36.1%. Under the High-performance and Low-latency profile, Adaptive showed a very small 0.3% increase in wire time compared with static `SecP384r1MLKEM1024`. Since this difference is very small, it may result from normal measurement variability rather than indicating a meaningful performance disadvantage.

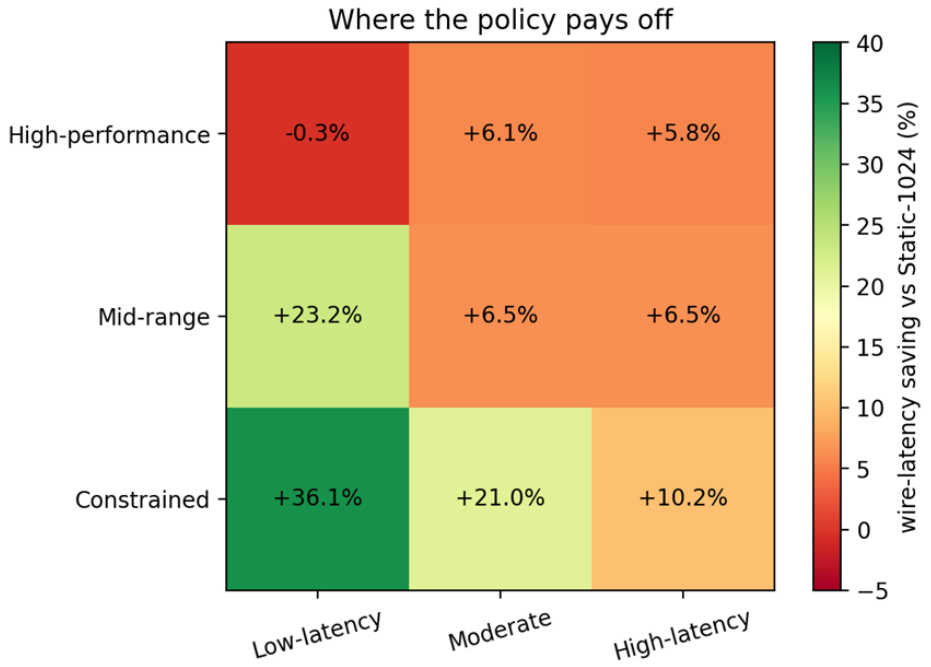


Figure 4.4: Wire-latency savings achieved by Adaptive relative to `SecP384r1MLKEM1024` across the nine tested device and network profiles.

As shown in Figure 4.4, the wire-latency savings ranged from −0.3% to 36.1%. The highest saving was recorded under the Constrained and Low-latency conditions (36.1%), followed by the Mid-range and Low-latency conditions (23.2%) and the Constrained and Moderate conditions (21.0%). Positive savings were recorded in eight profiles, whereas the High-performance and Low-latency profiles showed a slight negative saving

of −0.3%. In this profile, Adaptive and Static-1024 used the same hybrid configuration; therefore, this marginal difference is more likely attributable to normal run-to-run measurement variability than to a meaningful performance difference. Overall, the figure shows that the wire-latency savings varied according to the tested network and device-resource conditions.

Table 4.13: Adaptive Policy Compared with the Three Static Hybrid Configurations Across the Tested Device-Resource and Network Profiles

Device	Network	Adaptive selects	Adaptive wire (ms)	vs. Static-512		vs. Static-768		vs. Static-1024	
				ms	%	ms	%	ms	%
High-performance	Low-latency	`SecP384r1MLKEM1024`	53.7	8.8	19.5	8.7	19.4	0.2	0.3
High-performance	Moderate	`X25519MLKEM768`	148.6	0.7	0.5	0.1	0.1	−9.6	−6.1
High-performance	High-latency	`x25519_mlkem512`	260.2	0.1	0.0	−3.0	−1.1	−16.1	−5.8
Mid-range	Low-latency	`X25519MLKEM768`	44.9	0.0	0.0	−0.1	−0.2	−13.6	−23.2
Mid-range	Moderate	`X25519MLKEM768`	148.6	0.5	0.4	0.0	0.0	−10.3	−6.5
Mid-range	High-latency	`x25519_mlkem512`	260.2	−0.2	−0.1	−3.1	−1.2	−18.0	−6.5
Constrained	Low-latency	`x25519_mlkem512`	56.9	8.1	16.7	−0.5	−0.9	−32.2	−36.1
Constrained	Moderate	`x25519_mlkem512`	149.7	0.9	0.6	0.5	0.3	−39.9	−21.0
Constrained	High-latency	`x25519_mlkem512`	277.1	8.4	3.1	2.9	1.1	−31.5	−10.2

Note: Differences are calculated as Adaptive wire latency minus Static wire latency. Therefore, negative values indicate that Adaptive achieved lower wire latency, positive values indicate higher wire latency, and values close to zero indicate comparable wire latency.

Table 4.13 compares the Adaptive policy with the three static hybrid configurations across all device-resource and network profiles. The final Adaptive configuration varied according to the tested conditions. Under High-latency conditions, the final configuration was `x25519_mlkem512` across all device profiles. Under Moderate conditions, the final configuration was `X25519MLKEM768` on the High-performance and Mid-range devices, and `x25519_mlkem512` on the Constrained device. Under Low-latency conditions, the final configuration was `SecP384r1MLKEM1024` on the High-performance device, `X25519MLKEM768` on the Mid-range device, and `x25519_mlkem512` on the Constrained device.

Adaptive recorded lower wire latency than Static-1024 in eight of the nine tested profiles. The only exception was the High-performance and Low-latency profile, where Adaptive retained `SecP384r1MLKEM1024` and recorded 0.3% higher wire latency. However, Adaptive did not record the lowest wire latency relative to Static-512 and

Static-768 in every profile. Overall, the results show that the Adaptive configuration varied according to the combined network and device-resource conditions, providing a condition-dependent balance between the selected security level and the measured handshake costs, wire latency, CPU time, and handshake size, according to the available device resources.

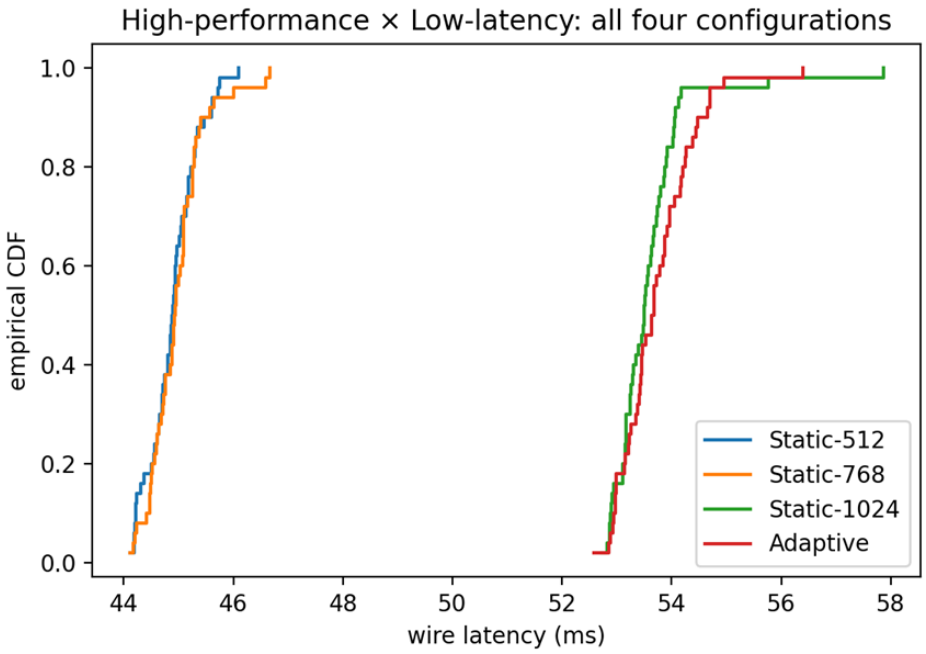


Figure 4.5: Empirical CDF of wire latency under the High-performance and Low-latency profile.

Figure 4.5 presents the empirical CDF of wire latency under the High-performance and Low-latency profile. Although `Static-512` and `Static-768` recorded lower wire latency, the Adaptive policy does not aim only to minimize wire latency. Under these favorable network and device-resource conditions, Adaptive retained `SecP384r1MLKEM1024` to provide the highest security level among the evaluated configurations, while accepting its additional wire-latency cost. Adaptive and `Static-1024` consequently showed closely similar distributions because both used `SecP384r1MLKEM1024`. This similarity supports the interpretation that the -0.3% difference reported in Figure 4.4 is marginal and most likely reflects normal run-to-run variability rather than a meaningful wire-latency difference. Therefore, the figure confirms that Adaptive

and `Static-1024` produced comparable wire-latency distributions when both used the same hybrid configuration.

4.11 Summary

This chapter evaluated AdaptiveQKE under the controlled device and network conditions defined in the experimental methodology. The validation checks did not identify failed trials, substantial execution-order drift, or inconsistencies between the enforced and classified device profiles. The adaptive policy produced the expected selection in all observed trials. Policy evaluation required minimal processing time, whereas RTT probing introduced a measurable, network-dependent first-connection cost. After group selection, the confidence intervals for the differences between AdaptiveQKE and the matching static configurations included zero. Therefore, the results provide no clear evidence of systematic post-selection latency overhead, although they should not be interpreted as proof of statistical equivalence.

The findings indicate that network conditions dominated absolute handshake latency and that the effect of handshake size increased as the available link rate decreased. The crypto-only measurements did not reveal a clear local latency difference between `x25519_mlkem512` and `X25519MLKEM768`, whereas `SecP384r1MLKEM1024` exhibited higher processing and transmission costs. Device constraints were also associated with greater latency increases for the heavier configurations, while the observed clustering around CFS scheduling boundaries was consistent with the effect of the enforced CPU quotas.

Within the evaluated conditions, AdaptiveQKE was generally associated with lower handshake costs than the strongest static configuration when a lighter group was selected, while retaining the strongest evaluated group under favorable conditions. It did not consistently outperform the lighter static configurations, since its purpose was to balance security level and performance rather than minimize latency alone. Accordingly, the results support the potential value of context-aware hybrid group selection within the tested prototype and experimental environment, without implying that the observed behavior will necessarily generalize to all TLS deployments.

Chapter 5

Conclusions and Future Work

5.1 Introduction

Endpoints running TLS 1.3 vary widely in available CPU and memory, and the networks connecting them vary just as widely in bandwidth and delay, yet a deployment typically fixes one hybrid key-exchange group for every connection regardless of this variation. AdaptiveQKE was built to test an alternative: a client-side layer that inspects the runtime context before each handshake and chooses among a fixed set of ML-KEM hybrid groups accordingly, leaving the TLS 1.3 exchange itself untouched. This chapter concludes the study by assessing the achievement of the research objectives and summarizing the main contributions of the proposed framework. It then discusses the practical implications of the work for hybrid-group deployment, identifies the limitations that define the scope of the conclusions, and outlines directions for future work before presenting the final concluding remarks.

5.2 Achievement of Objective 1

The first objective of this work was to design and implement a policy-driven mechanism that selects among ML-KEM and ECDH hybrid groups according to measured round-trip time and endpoint resource limits. The mechanism was intended to operate before

connection establishment without modifying the standardized TLS 1.3 handshake flow.

To address this objective, AdaptiveQKE was implemented as a client-side pre-handshake layer comprising a context probe, a two-stage policy engine, and a TLS client operating with an unmodified TLS 1.3 server. The experimental execution reported in Section 4.2 provides evidence that the prototype was operational across the evaluated conditions. Section 4.4 further shows that the observed group selections matched the outputs prescribed by the policy rules in all evaluated trials. This agreement supports the implementation fidelity of the policy engine, although it does not establish that the selected thresholds or resulting decisions are optimal beyond the tested conditions.

The post-selection comparisons in Section 4.5 did not provide clear evidence of a systematic wire-latency difference between AdaptiveQKE and the corresponding static configuration using the same group. This finding is consistent with the intended separation between group selection and handshake execution: the adaptive layer determines the group before the handshake, while the subsequent TLS exchange follows the behavior of the selected configuration. However, the absence of a detectable difference should not be interpreted as proof of statistical equivalence.

Accordingly, the implemented mechanism achieved the operational components of Objective 1 within the predefined allow-list, policy thresholds, device profiles, network conditions, and software environment used in this study. The result supports the feasibility of pre-handshake, context-aware group selection without altering the TLS 1.3 handshake flow, but it does not establish equivalent behavior across untested TLS implementations or deployment environments.

The conceptual Stage 0 security bounds described in Chapter 3 were neither implemented nor exercised experimentally. Objective 1 should therefore be regarded as achieved for the implemented RTT-driven and device-aware stages only. No empirical claim is made regarding application-defined security floors or ceilings, or resistance to attempts to influence the selection process through manipulation of the measured RTT.

5.3 Achievement of Objective 2

The second objective of this project was to quantitatively evaluate the effects of adaptive hybrid group selection on TLS handshake size, CPU time, and handshake latency by comparing it with fixed hybrid key-exchange configurations under controlled device and network conditions. This objective was addressed through nine device-network combinations, each evaluated using four configurations: Adaptive, Static-512, Static-768, and Static-1024. This resulted in 36 experimental conditions, each repeated for 50 trials, producing a total of 1,800 TLS handshake measurements. The corresponding evidence is presented in Sections 4.6-4.10, which examine the effects of network conditions, handshake size, local processing cost, device-resource constraints, and adaptive selection relative to the three static baselines. The results showed that network conditions had the greatest effect on wire latency, while differences among the hybrid groups became more visible on slower network links and constrained devices. Larger handshake messages incurred greater transmission costs as the link rate decreased, and heavier configurations showed greater latency increases under limited device resources. These findings demonstrate that the measured handshake costs depend on the operating context, supporting the need for adaptive selection. The comparison with the static baselines showed that AdaptiveQKE generally produced post-selection behaviour comparable to the static configuration using the same selected group. When the evaluated conditions caused the policy to select a lighter configuration, AdaptiveQKE reduced handshake costs relative to Static-1024, recording lower wire latency in eight of the nine tested profiles. Under the High-performance and Low-latency profile, AdaptiveQKE retained SecP384r1MLKEM1024 and produced a wire-latency distribution comparable to Static-1024. Overall, the baseline comparisons confirmed that AdaptiveQKE changed the selected security level according to the observed conditions, retaining the strongest group when conditions supported it and reducing handshake costs by selecting a lighter group under less favorable network or device conditions. Accordingly, Objective 2 was achieved within the controlled experimental scope of the project. The results demonstrate the condition-dependent effects of adaptive selection across the evaluated handshake metrics, rather than

claiming universal latency reduction relative to every static configuration.

5.4 Contributions

The research gap established throughout Chapter 2 and synthesized in Section 2.4 concerned the limited availability of context-aware mechanisms for selecting among ML-KEM and ECDH hybrid groups at the TLS pre-handshake level. The reviewed approaches primarily addressed static performance evaluation, algorithm-level optimization, or adaptive selection outside the TLS context. Against this background, the present work offers three distinct contributions: an artefact contribution, an empirical contribution, and a methodological contribution.

First, the artefact contribution is an operational prototype of a client-side, context-aware selection layer for hybrid TLS key exchange. AdaptiveQKE uses network and endpoint information to configure an evaluated hybrid group before connection establishment while preserving the standardized TLS 1.3 handshake flow and requiring no modification to the server. The prototype provides a concrete mechanism through which policy-driven hybrid-group selection can be studied within the TLS context. This contribution demonstrates feasibility in the evaluated OpenSSL and OQS Provider environment but does not imply production readiness or interoperability across all TLS implementations.

Second, the empirical contribution is a controlled characterization of the relationship between handshake size, available link rate, device constraints, and TLS handshake latency across the nine evaluated conditions. The combined use of networked measurements, residual-latency analysis, and a crypto-only phase provided a basis for distinguishing the effects of propagation, transmission, and local processing. The findings suggest that the performance implications of a hybrid group depend not only on its cryptographic parameters but also on the operational context in which it is used. This evidence extends the predominantly static or context-limited evaluations discussed in Chapter 2, while remaining conditional on the tested profiles and experimental environment.

Third, the methodological contribution is a reproducible and auditable procedure for evaluating adaptive hybrid-group selection. The experimental materials include frozen code hashes and dataset fingerprints, while a seeded interleaved execution order was used to limit configuration-specific drift. Medians and bootstrap confidence intervals were employed to report central tendency and uncertainty without relying on assumptions of normally distributed measurements. Furthermore, the relationship between configured delay and observed wire latency was examined before the propagation component was removed, allowing the latency decomposition to be based on an evaluated premise rather than an untested assumption.

Taken together, these contributions provide an implemented research artefact, empirical evidence concerning context-dependent hybrid-group performance, and a methodology for evaluating such mechanisms systematically. They address complementary aspects of the gap identified in Chapter 2, although their interpretation remains bounded by the prototype, configurations, and controlled conditions considered in this study.

5.5 Deployment Implications

The experimental findings provide several practical implications for deploying hybrid post-quantum key exchange in heterogeneous TLS environments. Rather than enforcing a single hybrid group across all connections, deployment decisions should consider both endpoint capability and network conditions before selecting the key-exchange configuration.

When network conditions are favorable and sufficient device resources are available, a stronger hybrid group can be retained without introducing unnecessary adaptation. In such environments, a static high-security configuration may already be suitable, and the benefit of adaptive selection may be limited. In contrast, when available bandwidth decreases or communication conditions become more restrictive, selecting a smaller hybrid group becomes more relevant because the transmission cost of larger handshake messages becomes increasingly significant.

The choice between lighter hybrid groups should also consider the source of the performance constraint. Where the dominant limitation is network capacity rather

than local computation, group selection should primarily reflect the communication cost associated with handshake size. However, under constrained endpoint resources, computational capability becomes an additional deciding factor. In such cases, selecting a lighter hybrid configuration can help avoid imposing unnecessary processing overhead on a resource-limited device.

The use of context probing should also depend on the expected connection pattern. For isolated or one-time connections, the additional pre-handshake probe may not be justified because its cost is paid entirely by a single connection. By contrast, when multiple handshakes are expected under similar device and network conditions, the probe cost can be amortized across subsequent connections, making adaptive selection more practical.

Accordingly, AdaptiveQKE is most appropriate for heterogeneous environments in which device capability and network quality vary enough for a fixed hybrid configuration to become inefficient. In relatively stable environments, a static configuration may remain sufficient, whereas adaptive selection is more useful when the operational context changes and a single group cannot provide the same balance between security requirements and performance across all connections.

5.6 Limitations

The scope boundaries fixed in Section 1.4, formal security proofs, modification of the TLS 1.3 protocol itself, and production deployment were declared out of scope before this work began and are not re-argued here. What follows are the boundaries that arose in executing the work.

Emulated network paths:

Network conditions were reproduced with `tc-netem` on a single host rather than measured over real WAN links. Because the shaped delay is applied at every crossing of the loopback relay, absolute latencies are inflated relative to a real network hop; comparisons between configurations hold, but the reported millisecond figures should not be read as production values.

Emulated device tiers:

Device tiers were instantiated with cgroup v2 CPU and memory limits on a single machine, not distinct hardware. This varies scheduling pressure only; it does not reproduce the instruction-set differences a real device of that class would have (e.g., a real mid-range phone's ARM NEON versus this host's AVX2/AVX-512, or a real constrained IoT device's lack of vector acceleration entirely), nor differences in cache or memory bandwidth.

Threshold circularity:

The adaptive policy's thresholds were chosen to align with the same device and network profiles used to test it. The 100% selection accuracy reported in Section 4.4.1, therefore, demonstrates that the policy correctly implements its decision rules, not that those thresholds generalize beyond the nine tested profiles.

Non-standard code point:

`x25519_mlkem512` uses a private-use TLS group code point rather than an IANA-assigned one, so this configuration is not yet an interoperable standard; results obtained with it may not transfer directly to a future standardized deployment of the same algorithm.

Stage 0 not implemented:

Stage 0 was designed but never implemented or exercised, so no claim of downgrade resistance is supported by this work: an adversary able to manipulate the observed RTT could in principle induce the policy to select a weaker group than the endpoint's true context would warrant.

Partial residual decomposition:

The residual-latency decomposition in Section 4.7 removes only the propagation component of wire latency; serialization delay and any retransmission effects remain folded into the reported residual and are not separated from it.

No formal security analysis:

No formal security analysis was carried out. The side-channel and availability motivations set out in Section 1.5 remain the motivation for this work, not a measured result of it.

5.7 Future Work

Future work can extend AdaptiveQKE in four main directions, each addressing a limitation identified in Section 5.6.

Completing the design:
Stage 0 should be implemented and evaluated: the security floor should be tested under manipulated RTT measurements to confirm the selected group cannot fall below the required minimum, and the upper bound evaluated separately as a control on higher-cost group use.

Broadening the evaluation:
Future evaluations should run AdaptiveQKE on physical devices with differing hardware and over real LAN, Internet, or mobile paths, to confirm the latency and CPU-time patterns observed here hold outside controlled emulation.

Improving the policy:
An RTT measurement could be cached and reused across connections instead of re-probing each time, with hysteresis added around thresholds to avoid group changes from minor RTT variation; testing across a wider range of networks and devices could replace the fixed prototype thresholds with empirically derived ones.

Security analysis:
The selection layer's resistance to a manipulated or inaccurate RTT signal should be evaluated directly, testing whether an attacker can force selection of a weaker group or higher resource use, and whether the policy continues to enforce the required security level under adversarial context.

5.8 Concluding Remarks

This work set out to determine whether a hybrid key-exchange group could be selected automatically, before the TLS 1.3 handshake begins, based on endpoint capability and network conditions, without modifying the protocol itself. The prototype selects correctly across every tested condition, and once a group is selected, its handshake

behaves indistinguishably from the matching static configuration; the measurable cost is a one-time, network-dependent delay incurred while acquiring that context. For deployment, the results argue against committing to a single static hybrid group: across all nine tested device-network conditions, AdaptiveQKE selected the configuration that best balanced security level against handshake cost for the conditions at hand, at the cost of that one-time first-connection delay.

Appendix A

Reproducibility and Supporting Materials

A.1 Purpose of the Appendix

This appendix provides supporting implementation and reproducibility information for the AdaptiveQKE experimental evaluation. The main experimental design, device and network profiles, policy thresholds, and performance results are presented in Chapters 3 and 4 and are therefore not repeated here. Instead, this appendix summarizes the software components used to conduct the experiments, the organization of the resulting datasets, and the implementation details required to trace the reported measurements through the experimental workflow.

A.2 AdaptiveQKE Implementation Components

The AdaptiveQKE prototype was implemented as a collection of Python modules, together with supporting shell scripts and OpenSSL configuration files. Each component was assigned a specific role in the experimental workflow.

Table A.1: Main AdaptiveQKE implementation components

Component	**Purpose**
`experiment.py`	Coordinates the complete experimental workflow, including server lifecycle, profile application, trial execution, CPU-time measurement, and result collection.
`policy_engine.py`	Implements the two-stage adaptive hybrid-group selection policy.
`context_probe.py`	Measures RTT and reads the available CPU share and memory limit from the active Linux cgroup v2 configuration.
`profile_ enforcement.py`	Applies device constraints through Linux cgroup v2 and network conditions through `tc-netem`.
`tls_client.py`	Executes individual TLS 1.3 handshakes using the selected hybrid group and records the client-side handshake timing.
`tls_server.py`	Runs the OQS-enabled OpenSSL TLS 1.3 server and the dedicated RTT probe server.
`metrics.py`	Measures wire-level TLS handshake latency and handshake size using a byte-counting TCP proxy.
`results.py`	Stores the measurements generated during the experiments, including handshake latency, wire latency, CPU time, and handshake size, in timestamped CSV files.
`generate_certs.sh`	Generates the self-signed certificate and private key used by the experimental TLS server.

The main experiment orchestrator executes fifty trials for each combination of three device profiles, three network profiles, and four evaluated configurations, resulting in 1,800 TLS handshake measurements. A fixed random seed controls the randomized execution order within each experimental condition.

A.3 Adaptive Selection Implementation

The adaptive policy was implemented as a deterministic two-stage selection procedure. In the first stage, the policy derives an initial network candidate from the measured round-trip time (RTT). For RTT values below 50 ms, the policy selects

`SecP384r1MLKEM1024`; for values above 100 ms, it selects `x25519_mlkem512`; and for intermediate values, it selects `X25519MLKEM768`.
In the second stage, the network candidate is validated against the detected device capability. A High-performance device retains the network-selected candidate, a Mid-range device is capped at `X25519MLKEM768`, and a Constrained device is restricted to `x25519_mlkem512`. Device classification is based on the available CPU share and memory limit, with the more restrictive of the two resource indicators determining the final device tier.

A.4 Experimental Profile Enforcement

Device-resource constraints were enforced using Linux cgroup v2. The implementation applies CPU quotas and memory limits to the client process, while the unrestricted High-performance profile is executed without these limits. A shortened 10 ms CFS scheduling period was used whenever a CPU quota was enforced (the Mid-range and Constrained profiles), to reduce artificially long scheduling stalls while preserving the intended CPU fraction.
Network conditions were enforced on the loopback interface using `tc-netem`. Delay, packet loss, and transmission rate were configured according to the selected network profile. The loopback MTU was also temporarily set to 1,500 bytes during the experiments to reproduce a standard Ethernet-sized transmission environment.

A.5 Context Collection and Performance Measurement

Before adaptive group selection, the context probe collects three runtime parameters: round-trip time (RTT), available CPU share, and memory limit. RTT is estimated using three concurrent TCP connection attempts to the dedicated RTT probe service. The first successful connection is used to estimate the RTT.

The device-side context is obtained directly from the Linux cgroup v2 configuration associated with the running client process. The CPU share is derived from the `cpu.max` control file, while the memory limit is obtained from `memory.max`. These raw values are converted into a CPU percentage and memory capacity in megabytes and are then supplied, together with the measured RTT, to the adaptive policy engine.

Three main performance metrics are collected during the experimental evaluation: TLS handshake latency, CPU time, and TLS handshake size. The TLS client records the handshake duration from the transmission of the ClientHello message to the completion of the TLS handshake. In addition, an external byte-counting TCP proxy implemented in `metrics.py` measures the authoritative wire-level latency and records the number of bytes exchanged in both the client-to-server and server-to-client directions.

CPU time is measured separately within the experiment runner using operating-system resource accounting. Using `resource.getrusage()`, the implementation records the user and system CPU time consumed by the client process and its child processes before and after each TLS handshake. The difference between these measurements represents the CPU execution time associated with the handshake. Because CPU time is based on processor usage rather than wall-clock duration, it remains distinct from the scheduling delays introduced by the enforced cgroup CPU quota.

The byte-counting proxy captures the transmitted and received byte counters when the TLS handshake is complete, before connection-shutdown traffic is included in the measurement.

A.6 Experimental Output

Experimental measurements are written to timestamped CSV files. The main dataset records the enforced and classified device profiles, available CPU share, memory limit, network profile, the network-only candidate group selected by Stage 1 for Adaptive trials, measured RTT, probing cost, policy-evaluation cost, evaluated configuration, trial number, execution order, requested hybrid group, handshake success, handshake latency, wire latency, CPU time, and handshake size measurements, including the client-to-server and server-to-client byte breakdown.

The main result file is generated using the naming format:

`results_YYYYMMDD_HHMMSS.csv`

A separate dataset is generated for the crypto-only evaluation. This dataset records the device profile, hybrid group, trial number, execution order, handshake latency, wire latency, handshake size, and execution success.
The crypto-only measurements provide additional information about the behavior of the evaluated hybrid groups under controlled device-resource conditions, independently of the emulated network profiles used in the main experimental evaluation.

A.7 TLS Experimental Configuration

The experimental TLS server operates on port 4433 using TLS 1.3 and is configured to support only the three hybrid groups evaluated in this study:

- `x25519_mlkem512`
- `X25519MLKEM768`
- `SecP384r1MLKEM1024`

Session tickets are disabled so that every experimental trial performs a complete TLS 1.3 handshake rather than relying on session resumption.
A separate TCP service operating on port 9000 is used for RTT probing. Separating the RTT probe service from the TLS server allows the context probe to perform a lightweight TCP connection measurement without initiating an additional TLS handshake.
A self-signed RSA-2048 certificate is generated for the experimental TLS server using the supporting `generate_certs.sh` script. The certificate provides server authentication within the controlled experimental environment, while the performance comparison itself focuses on the selected ML-KEM and ECDH hybrid key-exchange groups.

A.8 Integrity Verification

Table A.2 lists SHA-256 hashes of the frozen implementation files and the two generated datasets, supporting the auditability described in Section 5.4.

Table A.2: SHA-256 hashes of frozen implementation and dataset files

File	SHA-256
experiment.py	e842b21af4a547e51569a4665bfd5251bf48ec98d8d493a6f758e4ab78b3a46d
policy_engine.py	15060b1bbd6561ee20e73caef9dc08b853c6dadde19d2e943573173fea15038d
context_probe.py	06c144223c7a6b5b6324e9395185719b219d5a5b9717311c9d7f909e934e7ce8
profile_enforcement.py	7b8efbcf6ab52cd278a18ea201bea2a475460f5047bbf96ef7f8eb59cccf7886
tls_client.py	ca4a2dbf360413d62870686846142fcf281bc38828cc1ce2b32d36bbf28b4a37
tls_server.py	ba69d3796eb321895bb764e39fddf3ff2adc39b24fa482cdb43c77b429513cee
metrics.py	5c6a49f526b98c08d9970ea609b67af4d0f0b275f8e276f70f32cf7417b8bd4d
results.py	08501d9159bd4a23fe796819afbf5b55563c703aba76914c557f887c9e27ef02
generate_certs.sh	932c69b004271f80c7d673a3ee67aa703b08a60781dbdb3247fe2030f6f357ea
results.csv	95c6d106209b64bfb74a56aefa27deff44ba79e32670aed91cb72de9e1280eef
crypto_latency.csv	12895f130473a92a7222003557c16f9938f698cee66669066740820e9fcfd2cc

Bibliography

[1] European Union Agency for Cybersecurity (ENISA), "Post-quantum cryptography: Anticipating threats and preparing the future," ENISA, Tech. Rep., 2022, accessed: 2025. [Online]. Available: https://www.enisa.europa.eu/news/enisa-news/post-quantum-cryptography-anticipating-threats-and-preparing-the-future

[2] National Institute of Standards and Technology, "NIST releases first 3 finalized post-quantum encryption standards," National Institute of Standards and Technology, Tech. Rep., 2024, accessed: 2025. [Online]. Available: https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards

[3] T. Reddy and H. Tschofenig, "Post-quantum cryptography recommendations for TLS-based applications," Internet Engineering Task Force, Internet-Draft draft-ietf-uta-pqc-app-00, 2025, work in Progress. [Online]. Available: https://www.ietf.org/archive/id/draft-ietf-uta-pqc-app-00.html

[4] B. Westerbaan *et al.*, "Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3," Internet Engineering Task Force, Internet-Draft draft-ietf-tls-ecdhe-mlkem-04, 2025, work in Progress. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/

[5] Cloudflare, Inc., "Post-quantum cryptography · Cloudflare SSL/TLS documentation," 2024, accessed: 2025. [Online]. Available: https://developers.cloudflare.com/ssl/post-quantum-cryptography/

[6] Amazon Web Services, "Using hybrid post-quantum TLS with AWS KMS," 2024, accessed: 2025. [Online]. Available: https://docs.aws.amazon.com/kms/latest/developerguide/pqtls.html

[7] E. Rescorla, "The transport layer security (tls) protocol version 1.3," Internet Engineering Task Force, Tech. Rep., 2018, rFC 8446. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446

[8] N. Bindel, J. Brendel, M. Fischlin, and D. Stebila, "Hybrid key exchange in tls 1.3," International Association for Cryptologic Research (IACR), Tech. Rep., 2019, cryptology ePrint Archive, Report 2019/091. [Online]. Available: https://eprint.iacr.org/2019/091.pdf

[9] M. R. Albrecht *et al.*, "The impact of post-quantum cryptography on tls performance," International Association for Cryptologic Research (IACR), Tech. Rep., 2021, cryptology ePrint Archive. [Online]. Available: https://eprint.iacr.org/2021/123.pdf

[10] National Institute of Standards and Technology, "What is post-quantum cryptography?" 2024, accessed: 2025. [Online]. Available: https://www.nist.gov/cybersecurity/what-post-quantum-cryptography

[11] ——, "Module-lattice-based key-encapsulation mechanism standard," National Institute of Standards and Technology, Tech. Rep. FIPS 203, 2024. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.203.pdf

[12] A. Zafar and S. S. Iqbal, "Integrating code-based post-quantum cryptography into ssl/tls protocols through an interoperable hybrid framework," *Discover Computing*, vol. 28, p. 202, 2025. [Online]. Available: https://link.springer.com/article/10.1007/s10791-025-09735-7

[13] K. Souvatzidaki and K. Limniotis, "Post-quantum key exchange in tls 1.3: Further analysis on performance of new cryptographic standards," *Cryptography*, vol. 9, no. 4, p. 73, 2025. [Online]. Available: https://www.mdpi.com/2410-387X/9/4/73

[14] G. Tasopoulos, J. Li, A. P. Fournaris, R. K. Zhao, A. Sakzad, and R. Steinfeld, "Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems," in *Information Security Practice and Experience (ISPEC)*, ser. Lecture Notes in Computer Science, vol. 13620. Springer, 2022, pp. 432–451.

[15] P. Kampanakis, N. Childs-Klein *et al.*, "The impact of data-heavy post-quantum tls 1.3 on network performance," in *Fifth NIST Post-Quantum Cryptography Standardization Conference*. National Institute of Standards and Technology (NIST), 2024. [Online]. Available: https://csrc.nist.gov/csrc/media/Events/2024/fifth-pqc-standardization-conference/documents/papers/the-impact-of-data-heavy-post-quantum.pdf

[16] J. A. Montenegro, R. Rios, and J. López-Cerezo, "A performance evaluation framework for post-quantum TLS," *Future Generation Computer Systems*, vol. 175, no. 108062, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X25003577

[17] European Telecommunications Standards Institute (ETSI), "Unlocking digital transformation with autonomous networks," ETSI, Tech. Rep. White Paper No. 56, 2023, 1st edition. [Online]. Available: https://www.etsi.org/images/files/ETSIWhitePapers/ETSI-WP56_Unlocking-Digital-Transformation-with-Autonomous-Networks.pdf

[18] J. Zheng, H. Zhu, Y. Dong, Z. Song, Z. Zhang, Y. Yang, and Y. Zhao, "Faster post-quantum tls 1.3 based on ml-kem: Implementation and assessment," in *Computer Security – ESORICS 2024*, ser. Lecture Notes in Computer Science. Springer, 2024. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-70890-9_7

[19] P. Sengupta, M. Raikwar, S. Maharjan, F. Eliassen, and Y. Zhang, "Adaptive quantum-safe cryptography for 6g vehicular networks via context-aware optimization," in *Workshop on Security and Privacy of Next-Generation Networks (FutureG)*. San Diego, CA, USA: Network and Distributed

System Security Symposium (NDSS), February 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/futureg26-99.pdf

[20] Linode, "Post-quantum encryption with nginx on ubuntu 24.04," 2024, accessed: April 2026. [Online]. Available: https://www.linode.com/docs/guides/post-quantum-encryption-nginx-ubuntu2404/

[21] Open Quantum Safe, "Ml-kem algorithm details," 2024, accessed: 2026. [Online]. Available: https://openquantumsafe.org/liboqs/algorithms/kem/ml-kem.html

[22] QRAMM, "Ml-kem (kyber) explained," 2024. [Online]. Available: https://qramm.org/learn/ml-kem-kyber-explained.html

[23] IETF, "Composite kem for pq cryptography," 2024, internet-Draft. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-lamps-pq-composite-kem/

[24] Akamai, "Post-quantum cryptography implementation considerations in tls," 2023. [Online]. Available: https://www.akamai.com/blog/security/post-quantum-cryptography-implementation-considerations-tls

[25] The Linux Kernel Development Community, *Control Groups v2*, The Linux Kernel, 2024, accessed: 2025. [Online]. Available: https://docs.kernel.org/admin-guide/cgroup-v2.html

[26] S. Hemminger, *tc-netem(8) — Network Emulator*, The Linux Kernel, 2005. [Online]. Available: https://man7.org/linux/man-pages/man8/tc-netem.8.html